Set up for this session

last of backend.zip
ZIP

MarketIntelligence_Architecture_Design_Blue...rint_FA.docx
DOCX

این یه پروژه است که بر اساس یه پروژه بیسی ساخته شد که یه معماری داشت mediator,cqrs,ddd و موارد دیگه ای تو این پروژه بود
بعدا دوتا دامنه به این پروژه اضافه شد MarketIntelligence و Tools
دامنه MarketIntelligence با یه کم تغییر به این پروژه خورونده شد و در واقع یه جاهایی یه ترکیب اشتباهی از دوتا پروژه شده
حالا غیر از این دو تا دامنه بقیه پروژه سعی شده بده بر اساس معماری تمیزی درست بشه
الان یه فایل word هم کنار این پروژه برات فرستادم که یه سند هست برای تغییر این دامنه و تبدیل اش به دامنه ای متناسب با این پروژه سند ممکنه یه بخش هایی غیر دقیق باشه
هر جا سند با حرف هایی که الان دارم میزنم یکی نبود بگو تا چک کنیم
MarketIntelligence قراره دیتاهای بازار های مختلف رو بگیره و ذخیره کنه و در استراتژی ها و داشبورد ها و موارد دیگه ای از اینها استفاده کنه
Show more
مواردی که من مشکل دیدم تو این دامنه یکی scachule ای بود که خطا میخورد و یه جاهایی مشکل داشت
seed دیتا ها خیلی کثیف بود
سرویس های گرفتن دیتاها از سایت ها کثیف بود
وجود repository و اینترفیس ها تو این دامنه و موارد دیگه ای که تو سند هست
میخوام این دامنه بار طراحی بشه جوری که هم برای اضافه کردن بازار جدید برای گرفتن دیتاش خیلی راحت این اتفاق بیوفته و منعطف باشه هم بشه روی این دیتا ها به شکل های مختلف کوئری گرفت استراتژی های مختلف اجرا کردو یا دیتاهای دیگه ترکیب اش کرد و خروجی گرفت
مفهوم بود مشکل چیه و چی میخوام؟

Ran 7 commands, read memory, read a file

فایل‌ها رو باز می‌کنم و سند و ساختار دامنه MarketIntelligence رو می‌خونم تا جواب دقیق بدم.

بله، مشکل و خواسته‌ات رو فهمیدم. سند و سورس رو هم خواندم و بخش‌های اصلی سند رو با کد تطبیق دادم.

خلاصهٔ برداشت من

مشکل: MarketIntelligence یک پروژهٔ مستقل بوده که با کمی تغییر داخل پروژهٔ پایه ادغام شده. نتیجه‌اش:

- پنج پروژه‌ی جدا (Domain، Application، Collectors، Persistence، Infrastructure) با وابستگی‌های درهم.
- دو مسیر موازی برای کندل و دو مفهوم `IEntity` .
- زمان‌بند ناپایدار.
- seed و تنظیمات پراکنده.
- Collectorهایی که خطا را می‌بلعند و لیست خالی برمی‌گردانند.
- Repository و اینترفیس‌های بی‌دلیل.

خواسته: بازطراحی کل دامنه با سه هدف:

1. اضافه‌کردن بازار یا Provider جدید ساده باشد (یک Provider بنویسی، ثبت کنی، تمام).
2. روی داده بشود به شکل‌های مختلف Query گرفت.
3. روی داده استراتژی اجرا شود، با دادهٔ دیگر ترکیب شود و خروجی بدهد.

نتیجهٔ تطبیق سند با کد

این موارد درست بودند:

- وابستگی‌های پروژه‌ها (Application و Collectors به Domain، Infrastructure بدون reference).
- باگ `StopAsync` که `base.StartAsync` صدا می‌زند.
- `IsRuntime` که با `LastRunSucceed == false` شرط Period را دور می‌زند.
- ثبت دوبارهٔ `IDataFetcher` و `ICandleSourceFactory` .
- ثبت `ICandleSource` در دو جا. `RegisterCandleSources` کل AppDomain را اسکن می‌کند.
- حذف کندل‌ها قبل از دریافت دوباره: `FetchWeeklyCandles` و `CandleBulkService` (حذف ۱۰ کندل آخر).
- ثبت entityها با Reflection و نام رشته‌ای `IEntity` در `FundDbContext` .
- ۱۶ جدول در snapshot.

جاهایی که سند با حرف‌های تو یکی نیست یا کم دارد:

1. **دامنهٔ سند محدودتر از خواستهٔ توست.** سند فقط مسیر داده (کندل، وضعیت جاری، صف) را درست می‌کند و Analysis و Strategies را به «فاز ۹» می‌اندازد. ولی خواستهٔ تو یک طراحی یکپارچه است که لایهٔ Query، ترکیب داده و خروجی هم دارد.
2. **«افزودن آسان بازار» در سند طراحی نشده.** فقط اسم‌ها و کلیدها آمده. الان `MarketType` یک enum است و هر بازار جدید یعنی دست‌زدن به Source، Collector، seed، enum و DI. سند این مدل را عوض نکرده و نگفته Provider چطور plug-in شود.

3. «Query، **ترکیب و خروجی» در سند نیست.** فقط ۹.۱ یک خط درباره Query محدود به نماد و بازه دارد. چیزی دربارهٔ ترکیب دو دیتاست یا خروجی گرفتن نیست.
4. **مصرف‌کنندهٔ دیگری هم هست.** سند فقط `StrategyController` را نام می‌برد. در سورس هم بیرون از MarketIntelligence فقط همان کنترلر، `Program.cs` و `FundDbContext` به آن وابسته‌اند. ولی تو از داشبوردها هم حرف زدی و اینجا خبری از آن‌ها نیست. باید بدانیم داشبورد کجاست.
5. `FundDbContext` . سند می‌گوید حفظش کن، و درست است که الان همین هست. اما چون اسم solution هم MarketAnalyzer است، باید مطمئن شویم این قید هنوز مدنظرت هست. فقط در سورس، رفرنس‌های Fund/Bourse/Assets در Api و Reports هم هست که به این دامنه ربطی ندارند.
6. **چیزهایی که سند عمیق نبررسی کرده:** `Financial/` در Infrastructure (حدود ۲۰۰۰ خط Codal mapping)، `CodalStatementResponse` با ۱۵۴۰ خط، `MarketCollector` و `CandelClientFactory` قدیمی، و ۹ `CandleSource` (Ime، Kucoin، Nobitex، Wallex، Tgju، Kilid، Nahayatnegar، Tse، TseIndex) که هر کدام الگوی متفاوتی دارند.
7. **دامنهٔ** Tools هیچ ارتباطی با MarketIntelligence ندارد (Note، Bookmark، Password). از بازطراحی بیرون است.

سؤال‌هایی که طراحی را تعیین می‌کنند

1. کندل‌ها و داده‌های جاری برای بک‌تست و استراتژی لازم است یا فقط آخرین وضعیت؟ (یعنی snapshot لحظه‌ای هم نگه داریم؟)
2. حجم واقعی: تعداد نماد و کندل، و هر چند وقت یک بار دریافت؟
3. "ترکیب با داده‌های دیگر" یعنی چه؟ مثلاً ترکیب قیمت بازار با داده‌های داخلی صندوق، یا ترکیب چند بازار با هم؟
4. خروجی چه شکلی است: JSON برای داشبورد، Excel، یا هر دو؟
5. آیا Providerها (Nobitex، Kucoin، ...) هنوز همه لازم‌اند یا بعضی را می‌شود کنار گذاشت؟

اگر تأیید می‌کنی، قدم بعدی من این است: **دو بخش اول سند (به‌خصوص ۷ و ۸) را با طراحی «Provider plug-in + لایهٔ Query/Dataset» تکمیل کنم** و برای تأیید بیاورم، پیش از دست‌زدن به کد. فعلاً هیچ کدی را تغییر ندادم.

این قسمت رو یه توضیح بدم
حذف کندل‌ها قبل از دریافت دوباره: `FetchWeeklyCandles` و `CandleBulkService`
(حذف ۱۰ کندل آخر).
تو گرفتن دیتا آخرین دیتا ممکنه همین امروژ باشه یعنی روزی که هنوز تموم نشده وچون امروز تغییر میکنه آخرین دیتا رو که حالا محتاتانه 10 روز پیش بود رو حذف میکنم دوباره میشونم ولی ما اگه امروز رو نگیریم و امروز رو جای دیگه ای ذخیره کنیم دیگه شاید نیاز به پاک کردن اصلا نباشه (این رو

پیشنهادات رو بگو)
در مورد گرفتن دیتاها هم چون برای بازار ایران کندل های تعدیل شده رو میگیرم و کندل ها با افزایش سرمایه و سود دوباره تعدیا میشن و من تاریخ افزایش سرمایه و سود رو ندارم و وقتی تعدیا میشن کل دیتای اون سهم کل کندل ها عوض میشه
از زمان تعدیل به قبل اش برای همین من کل دیتا رو مثلا هفتگی پاک میکنم و همه دیتا رو دوباره میگیرم که میدونم هزینه اضافه و اشتباه هست مثلا میشه 10 روز آخر رو چک کرد اگه مغایرت نداشت اون سهم رو حذف نکرد و اگه مغایرت داشت همه دیتای اون سهم رو حذف کرد (اگه پیشنهاد دیگه ای داری بگو)

در مورد این
`Financial/` در Infrastructure (حدود ۲۰۰۰ خط Codal mapping)،
`CodalStatementResponse`
فعلا این قسمت رو برای گرفتن دیتا کاری نداشته باش بعدا یادت باشه بریم سراغ اش این قسمت اصلا درست پیاده نشده
تو بحث گرفتن دیتا فعلا اینها رو داریم
`MarketCollector` و `CandelClientFactory` قدیمی، و ۹ `CandleSource` (Ime، Kucoin، Nobitex، Wallex، Tgju، Kilid، Nahayatnegar، Tse، TseIndex) که هر کدام الگوی متفاوتی دارند.
که باید بازطراحی بشن

در مورد
دامنهٔ Tools هیچ ارتباطی با MarketIntelligence ندارد (Note، Bookmark، Password). از بازطراحی بیرون است.
هم الان کاری ندارم بعد اش تموم شدن این قسمت اون هم بررسی میکنیم

در مورد سوالات
1- ممکنه داده های جاری هم موثر باشه و نقطه ورود رو اون تعیین کنه
2-حجم دیتا برای هر نماد روزانه شاید 100 کندل که غیر از اون 10 روز رو دوباره نمینشوندیم و دریافت روزانه
برای اینکه پاک کنیم و دوباره بشونیم همه دیتاش شاید 10 سال و دریافت هفتگی تو کدمون ولی شاید بتونیم همین رو تو قالب روزانه ببریم (یعنی اگه مغایرت بود کلدیتای سهم پاک بشه و دیتای کلی سهم گرفته بشه)
3- الان من دیتهای بازارهای مختلف رو میگیرم مثلا نمودار بیتکوین رو با فولاد میخوام مقایسه کنم که تو دوتا بازار مختلف هستن
4-الان خروجی رو کاری ندارم شاید excel بهتر باشه این قسمت تو کل فرایند میشه آخری
5-اگه منظورت اینه همه بازار ها لازم هست که آره همه بازار ها رو میخوام ولی اگه منظورت کد های هر کدومه من هدفم گرفتن دیتاست فرقی نمیکنه

اما یه توضیح دیگه بدم
اینجا قراره دوتا اتفاق بیوفته
1- دیتا ها دریافت بشن
2- از اون ها استفاده بشن
الان هر دو قسمت داره انجام میشه ولی معماری ها خیلی تو در تو و کثیف هستن
الان مفهوم هست موصوع؟

Show more

Added to memory

بله، مفهوم شد. دو سمت داریم که الان توی‌هم هستند و باید از هم جدا شوند:

1. **دریافت (Ingestion):** Provider → نرمال‌سازی → ذخیره.
2. **استفاده (Consumption):** Query، ترکیب دیتاست‌های چند بازار، استراتژی، داشبورد و در آخر خروجی (احتمالاً Excel).

این دو سمت فقط از طریق مدل ذخیره‌شده و یک قرارداد خواندن به هم وصل شوند. سمت استفاده هیچ‌وقت به Collector وابسته نباشد.

پیشنهاد دربارهٔ کندل امروز

کندل امروز را اصلاً داخل جدول تاریخی نریز. یک مدل جاری جدا بگذار، مثلاً `CurrentCandle` با کلید (بازار، نماد، تایم‌فریم). هر بار دریافت، همان رکورد Upsert شود. بعد از پایان روز و دریافت نهایی، همان رکورد به تاریخی منتقل شود.

نتیجه: جدول تاریخی فقط کندل‌های نهایی دارد و دیگر حذف «۱۰ کندل آخر» لازم نیست. این با نیاز «داده جاری» هم یکی می‌شود. برای کریپتو که ۲۴ ساعته است، «پایان روز» را با مرز UTC یا تهران مشخص کن.

پیشنهاد دربارهٔ تعدیل

ایدهٔ تو (چک آخرین چند روز) درست است. فقط می‌شود ساده‌ترش کرد.

کشف تعدیل: تعدیل ضریبی است که روی همهٔ کندل‌های قبل از رویداد اعمال می‌شود. پس اگر هر کندل نهایی قبلی (مثلاً دیروز) با دریافت جدید فرق کرد، یعنی تعدیل شده. لازم نیست ۱۰ روز را چک کنی؛ چند نقطه کافی است: آخرین کندل ذخیره‌شده، یک کندل حدود ۳۰ روز قبل و کندل اول. مقایسه روی قیمت‌ها (OHLC) با tolerance انجام شود.

دریافت دوباره: فقط برای نمادی که مغایرت دارد، و به این شکل:

1. کل تاریخچه دریافت شود (در حافظه).
2. اعتبارسنجی شود (تعداد و بازه).
3. در یک تراکنش جایگزین شود.

هیچ‌وقت قبل از موفقیت دریافت چیزی حذف نشود. اگر دریافت شکست خورد، داده قبلی می‌ماند. این با تغییر Collector که شکست را از «لیست خالی» تشخیص دهد کامل می‌شود.

روزانه چک کن، نه هفتگی. چون چک ارزان است، تعدیل فردا صبح اعمال می‌شود و هفته‌ها منتظر نمی‌مانی. فقط نمادهای مغایر تاریخچهٔ کامل را دوباره می‌گیرند.

ضریب تعدیل را ذخیره کن: نسبت دو close (قبلی و جدید) یعنی ضریب. اگر در جدولی مثل `CorporateAdjustmentLog` ثبت شود (نماد، تاریخ کشف، ضریب)، تاریخ افزایش سرمایه و سود را که نداشتی به‌تدریج به دست می‌آوری. با آن می‌توانی بعداً تحلیل کنی، یا نمادهایی که تعدیل شدند را روی داشبورد نشان بدهی.

اعمال ضریب مستقیم روی کندل‌های ذخیره‌شده (بدون دریافت دوباره) ممکن است به‌خاطر رُند شدن قیمت‌ها با مقدار منبع کمی فرق کند، پس پیشنهاد نمی‌کنم.

یک شبکهٔ ایمنی: هر ماه یک بار یک نمونه از نمادها را کامل دوباره بگیر و مقایسه کن. اگر چیزی از چک سریع جا افتاده بود، همین‌جا پیدا می‌شود.

یک سؤال و یک نکته

- **سؤال:** آیا منبع TSE کندل غیرتعدیل‌شده هم می‌دهد؟ اگر بدهد، بهترین طرح این است که خام را ثبت کنیم و تعدیل را خودمان با ضریب‌های ثبت‌شده اعمال کنیم. آن‌وقت دیگر هیچ دریافت دوباره‌ای لازم نیست. باید روی خود API چک کنیم.
- **نکته:** مقایسهٔ بیتکوین با فولاد دو چیز می‌خواهد که باید در مدل ذخیره بیاید. یکی واحد پول (دلار، تومان یا ریال)، دیگری تقویم معاملاتی: کریپتو هر روز و بورس فقط شنبه تا چهارشنبه معامله دارد. بدون این‌ها هم‌راستاسازی محور زمان سخت می‌شود. بهتر است از همین الان در مدل `Market` و `Symbol` فیلد `Currency` و `TradingCalendar` داشته باشیم.

ذکر «نقطهٔ ورود را داده جاری تعیین کند» را دقیق نفهمیدم. منظورت این است که داده جاری مشخص کند کدام نمادها را دریافت کنیم (مثلاً فقط نمادهای فعال)، یا نقطه‌ای که استراتژی از آن شروع می‌کند؟ بگو تا طراحی را بر همان بنا کنم.

اگر موافقی، قدم بعد: طراحی **مدل Provider** (یک قرارداد برای همهٔ ۹ منبع و افزودن بازار جدید فقط با نوشتن یک کلاس و یک ثبت) و مدل داده جاری/تاریخی/تعدیل. هنوز هیچ کدی را تغییر نمی‌دهم.

در مورد سوال ها
درحال حاضر من دیتا رو دارم از جایی میخونم که تعدیل شده است
اگر مثلا از tse گرفتم که تعدیل نشده است باید دیتاهای مربوط به افزایش سرمایه رو هم بگیرم تا بتونم تعدیل کنم
و این رو هم ببین تو پروژه یعنی این فرض رو بزار که ممکنه این اتفاق بیوفته و تو معماری سیستم باید چه کار کنیم (احتمالا گرفتن اطلاعات خیلی معماری اش عوض نشه ولی تو استفاده از دیتا اول باید تعدیل بشه بعد استفاده بشه)
در مورد نقطه ورود : من این رو تو جواب این دادام
کندل‌ها و داده‌های جاری برای بک‌تست و استراتژی لازم است یا فقط آخرین وضعیت؟ (یعنی snapshot لحظه‌ای هم نگه داریم؟)
شاید درست نفهمیدم
اگه منظورت اینه که هر دفعه که دیتای جاری رو میگیریم به دیتای قبلی جاری هم نیاز داریم، نه به این دیتا نیاز نداریم و دیتای مثلا قیمت الان بیت کوین 83 هست دوباره دیتا رو گرفتیم شد 84 اون 83 رو نیاز نداریم
و با provider موافقم
فقط
اگر کلاسی اضافه، حذف یا تغییر کرد ترجیح میدم کل اون کلاس ها رو در مسیر پروژه بهم بدی و من کپی کنم تو پروژه و replace بشه
و اگه یه سری قرار شد پاک بشه و زیاد بودن با powershell یه کد بده بزنم و همه حذف بشه
سوال دیگه ای داری؟

Show more

Updated 2 memories

باشه، روش تحویل رو می‌پذیرم: کلاس‌ها کامل و با مسیر پروژه می‌دم و برای حذف‌ها اسکریپت PowerShell می‌نویسم.

اثر دو جوابت روی معماری

داده جاری: هر کلید یک رکورد دارد (مثلاً بازار، نماد) و با هر دریافت موفق همان رکورد Upsert می‌شود. هیچ تاریخچه‌ای برایش نگه نمی‌داریم.

تعدیل: هر Provider اعلام می‌کند داده‌اش چه وضعی دارد: `AlreadyAdjusted` یا `Raw`. بر این اساس دو مسیر داریم:

- `AlreadyAdjusted` **(وضع فعلی):** دریافت همان چیزی است که گفتیم، همراه با کشف مغایرت و دریافت دوباره فقط برای نماد مغایر.
- `Raw` **:** کندل خام ذخیره می‌شود، و رویدادهای افزایش سرمایه و سود هم در جدولی جدا (`CorporateAction` : نماد، تاریخ، نوع، مقدار) از Providerی که آن‌ها را می‌دهد. دریافت رویدادها خودش یک Provider دیگر است.

در سمت استفاده فقط یک نقطهٔ خواندن وجود دارد. این نقطه هر سری را قبل از تحویل به استراتژی و داشبورد تعدیل می‌کند: کندل خام را می‌خواند، ضریب تجمعی رویدادها را حساب می‌کند و اعمال می‌کند. در نتیجه استراتژی‌ها هیچ‌وقت داده خام را نمی‌بینند و Provider جدید بدون تغییر کد استراتژی اضافه می‌شود.

یک ریسک: فرمول تعدیل TSETMC دقیقاً با فرمول‌های رایج یکی نیست. اگر روزی مسیر `Raw` را روشن کنیم، باید خروجی را روی چند نماد با داده تعدیل‌شدهٔ فعلی مقایسه کنیم.

سؤال‌هایی که هنوز دارم

1. **تایم‌فریم‌ها:** آیا هفتگی و ماهانه را از روی روزانه بسازیم و دیگر جدا از منبع نگیریم؟ این کار حجم دریافت را کم می‌کند و دو سری ناهماهنگ نمی‌شوند. و آیا کندل درون‌روزی (۱ دقیقه، ۱ ساعته) لازم است؟
2. **مقایسهٔ بازارها:** منظورت از مقایسهٔ بیتکوین و فولاد این است که هر دو را روی یک مبنا (مثلاً شروع = ۱۰۰ یا درصد تغییر) ببینی، یا تبدیل واقعی ارز (بیتکوین به تومان)؟ اگر دومی، نرخ دلار هم باید به‌عنوان یک سری دادهٔ دیگر در همین سیستم باشد. من فرض می‌کنم مبنای ۱۰۰.
3. **جدول‌ها:** کندل‌ها را هر وقت بخواهیم می‌شود دوباره گرفت. آیا مشکلی دارد که جدول‌های جدید از صفر ساخته شوند و دادهٔ قدیمی را Migration نکنیم و فقط دوباره دریافت کنیم؟ این Migration را خیلی ساده می‌کند. فقط جدول‌های واقعاً دستی (صندوق‌ها و seed نمادها) نیاز به انتقال دارند.
4. **ترتیب کار:** پیشنهادم این است که اول سند طراحی کوتاه برای تأیید بیاورم (لایه‌ها، قرارداد Provider، مدل داده، قرارداد خواندن)، بعد کد را مرحله‌ای بدهم. موافقی، یا مستقیم برویم سراغ مرحلهٔ اول کد؟

با جواب این‌ها شروع می‌کنم.

در مورد سوال1 این که گفتی هفتگی و ماهانه رو از روزانه بسازی عالیه
در مورد 1 دقیقه و 1 ساعته هنوز نمیگیرم و استرلتژی روش ندارم ولی احتمالاش هست اضافه بشه تو معماری چه کاراش کنیم؟
2 - مثال زدم مثلا میخوام نسبت رشد هاشون رو مقایسه کنم هم نسبت واقعی خودشون هم نسبت ریالی
تو اولی بدون در نظر گرفتن دلاره و دومی با دلار

ولی این موضوع میره تو بحث استراتژی ها و الان تو این مرحله ما میخوایم گرفتن دیتا ها رو درست کنیم
3- اتفاقا در مرود دامنه MarketIntelligence ترجیح میدم همه جداول تغییر کنه چون هم اسامی نمیخوره باهم و هم کار رو راحت تر میکنه
کل دیتا دوباره گرفته میشه و مشکلی نیست اونهایی هم که قراره دیتاش وجود داشته باشه همه تو seed هست که اون قسمت رو هم باید فکری بکینم
بنابراین حذف دیتای این بخش مشکلی نداره
4-موافقم
یه سند داشته باش و براساس اون چند فاز کن و بعد شروع کنیم

Show more

Make doc changes (batch)

سند اولیه آماده است؛ الان بخش‌ها رو یکی‌یکی پر می‌کنم.

Ran 2 commands, used Claude Docs integration, used a tool

الان بخش اول سند (هدف و دامنه) رو پر می‌کنم.

Update content

بخش دوم: تصمیم‌ها و اختلاف با سند قبلی.

Update content

بخش چهارم: مدل Provider.

Update content

بخش پنجم: مدل داده.

Update content

بخش ششم: جریان دریافت.

Update content

بخش هفتم: تعدیل.

Ran a command, used Claude Docs integration

بخش‌های هشتم و نهم.

Used Claude Docs integration

This response didn’t load.

در مورد سوال1 این که گفتی هفتگی و ماهانه رو از روزانه بسازی عالیه
در مورد 1 دقیقه و 1 ساعته هنوز نمیگیرم و استرلتژی روش ندارم ولی احتمالاش هست اضافه بشه تو معماری چه کاراش کنیم؟
2 - مثال زدم مثلا میخوام نسبت رشد هاشون رو مقایسه کنم هم نسبت واقعی خودشون هم نسبت ریالی
تو اولی بدون در نظر گرفتن دلاره و دومی با دلار
ولی این موضوع میره تو بحث استراتژی ها و الان تو این مرحله ما میخوایم گرفتن دیتا ها رو درست کنیم
3- اتفاقا در مرود دامنه MarketIntelligence ترجیح میدم همه جداول تغییر کنه چون هم اسامی نمیخوره باهم و هم کار رو راحت تر میکنه

کل دیتا دوباره گرفته میشه و مشکلی نیست اونهایی هم که قراره دیتاش وجود داشته باشه همه تو seed هست که اون قسمت رو هم باید فکری بکینم
بنابراین حذف دیتای این بخش مشکلی نداره
4-موافقم
یه سند داشته باش و براساس اون چند فاز کن و بعد شروع کنیم

Show more

سند تقریباً نصفه‌ست و بخش‌های ۹ تا ۱۱ و معماری هدف مونده؛ ادامه می‌دم. الان بخش زمان‌بندی، تنظیمات و Seed.

Read 4 files, updated memory, used Claude Docs integration

سند طراحی ingestion نسخه ۲ آماده است و داخل پنل باز شده. می‌تونی بخونی، مستقیم داخلش ویرایش کنی یا کامنت بذاری.

سند این بخش‌ها رو داره: دامنه، تصمیم‌ها و اختلاف با سند قبلی، معماری هدف، مدل Provider، مدل داده، جریان دریافت، تعدیل، قرارداد خواندن، زمان‌بندی/تنظیمات/Seed، و فازهای ۰ تا ۷ با معیار پذیرش و Rollback.

قبل از شروع فاز ۰ و ۱ این سه تا رو باید جواب بدی:

1. **نقطهٔ ورود:** منظورت از «داده جاری نقطهٔ ورود رو تعیین می‌کنه» دقیقاً چیه؟ (مثلاً فقط نمادهایی که امروز معامله داشتن کندل بگیرن؟)
2. **Nobitex و Wallex:** هر دو بازار CryptoToman رو پوشش می‌دن. پیشنهادم یکی اصلی با `Priority` و دومی fallback باشه. موافقی؟
3. **UI و API‌ها:** بیرون از backend، کدوم داشبورد یا UI از این دامنه استفاده می‌کنه؟ توی کد فقط `StrategyController` دیدم.

هنوز هیچ کدی عوض نشده. وقتی تأیید کردی، فاز ۱ رو شروع می‌کنم: کلاس‌های کامل با مسیر پروژه به‌صورت zip.

1-منظور خاصی نداشتم، سوال ات رو اشتباه فهمیده بودم.
2- ما دوتا دیتای یکسان برای یک نناد نداریم ممکنه دو منبع یکسان داشته باشیم که بخوایم بین اشون سوییچ کنیم
ما مثلا bitcointoman رو از یکی از دو منبع میگیریم نه از هر دو ممکنه برای دیتای بورس هم منبع

جدیدی تعریف بشه ولی یکی اش رو میگیریم
3-عمدا ui رو نفرستادم چون اون هم باید تغییر کنه بعدا
وقتی به بحث استراتژی ها رسیددیم بگو اون رو هم بفرستم ومشکلات اش رو بگم که بعد از بک اند بریم سراغ اون
چند تا سوال

ها حذف میشن Setting
ها چی میشن Seed
بزنم و بسازم و این دیتا اولیه باید وجود داشته update من ممکنه بعدا دوباره بخوام دیتابیس رو باشه
ها تغییر کنه و باید جایی باشه تغییراشون بدم - یه پیشنهاد میتونه. یه قسمت setting و ممکنه این باشه که اونجا تغیر کنه ولی باید جایی ذخیره بشه که تو بالا اومدن اولیه هم تغییر کرده باشه ui
سوال دیگه ای هست؟

Show more

جواب‌هات رو در سند اعمال می‌کنم: انتخاب منبع فعال به‌جای fallback، تنظیمات قابل‌تغییر در دیتابیس، و Bootstrap بعد از ساخت دیتابیس.

Used Claude Docs integration

سند رو با جواب‌هات به‌روز کردم.

- **منبع داده:** برای هر بازار در هر زمان فقط یک Provider فعال است (مثل bitcointoman یا بورس). تعویضش یک تغییر تنظیم است، نه کد. بعد از تعویض، تاریخچهٔ آن بازار از منبع جدید دوباره گرفته می‌شود و فقط بعد از دریافت موفق جایگزین می‌شود. `Priority` و fallback از سند حذف شد.
- **UI و استراتژی‌ها:** UI عمداً بعد از بک‌اند می‌رود. وقتی به فاز استراتژی‌ها رسیدیم، خودم می‌گم کدش رو بفرستی و مشکلاتش رو بگی.

جواب سؤال‌هایت

تنظیمات سه نوع شد، هر کدام یک جا:

- **راز** (API key، connection string): فقط User Secrets یا Environment.
- **پیش‌فرض‌های کد** (URL پیش‌فرض، retry، Period): `Options` داخل کد هر Provider.

- **قابل‌تغییر توسط تو** (منبع فعال هر بازار، URL، فعال/غیرفعال و Period هر job): جدول `MarketSettings` در دیتابیس.

اعمال می‌شود و در بالا آمدن بعدی هم همان مقدار ذخیره‌شده خوانده restart ها هر بار از همین جدول می‌خوانند، پس تغییر بدون Job بعداً UI خواندن و نوشتن تنظیمات ساخته می‌شود و API می‌شود. فعلاً فقط.

یک scheduler یا ساخت دوبارهٔ دیتابیس، قبل از شروع Migration **ها** حذف نمی‌شوند، شکلشان عوض می‌شود. بعد از هر Seed `MarketBootstrapService` idempotent انجام می‌دهد: سه کار را یک‌بار و

1. `Markets` را از Descriptor Providerها می‌سازد.
2. تنظیمات پیش‌فرض را فقط وقتی وارد می‌کند که کلیدشان در جدول نباشد، پس مقدار ویرایش‌شدهٔ تو هرگز overwrite نمی‌شود.
3. فهرست‌های ثابت نماد (Ime، TseTmcIndex، RealEstateRegion، CryptoToman، Macro) و نوع صندوق‌ها را از فایل embedded هر Provider وارد می‌کند. این فایل‌ها JSON همین seedهای فعلی‌اند که به‌جای پوشهٔ bin داخل assembly قرار می‌گیرند.

فهرست‌های زنده (Kucoin، Tse، صندوق‌ها) را jobهای کاتالوگ بعد از شروع می‌گیرند. چون همه‌چیز Upsert است، چندبار اجرا شدن اثری ندارد.

یک نکته: چند خانهٔ جدول تصمیم‌های باز و فاز ۱ بعد از ویرایش، فونت کد گرفته‌اند. محتوا درست است و فقط ظاهرشان را بعداً صاف می‌کنم.

سؤال دیگری ندارم. اگر سند رو تأیید کنی، فاز ۰ و ۱ رو شروع می‌کنم. فاز ۰ فقط یک چک‌لیست برای ثبت وضعیت فعلی است. فاز ۱ قراردادهای Provider، موجودیت‌ها و EF config نه جدول جدید است و به‌صورت zip با مسیر پروژه و XML comment تحویل می‌دهم. اگر نام پروژهٔ `Collectors` رو به `Providers` تغییر بدیم هم باید بگی؛ سند همین رو فرض کرده.

ادامه بده

Ran 8 commands, read memory

This response didn't load.

ادامه بده

Ran 3 commands, shared files

فاز ۱ تحویل شد: `MarketIntelligence-Phase1.zip` .

این کد کامپایل نشده است. در محیط من `dotnet` نیست و دانلودش بسته است. کد را فقط با بازبینی دستی چک کرده‌ام. اولین کار شما `dotnet build` است. هر خطایی دیدید برایم بفرستید تا درست کنم.

محتوای zip

- **Domain:** ۹ موجودیت و ۵ enum در `Entity/MarketData/` .
- **Persistence:** ۹ کلاس EF Configuration در `MarketDataModelConstants` به‌علاوه `Context/MarketData/` . ایندکس‌های unique همان لایهٔ ایمنی دیتابیس هستند.
- **Application:** قراردادهای Provider که `FetchResult<T>` (از جمله `MarketData/Providers/` در `MarketData/Reading/` شکست منبع را از «داده خالی» جدا می‌کند) و قراردادهای خواندن در .
- فقط فایل جدید اضافه می‌کند و چیزی را بازنویسی نمی‌کند. فایل‌ها را روی مسیر پروژه کپی کنید.

فاز ۰ (قبل از Migration)

1. یک branch جدید بسازید.
2. از دیتابیس backup بگیرید.
3. وضعیت فعلی build و تست‌ها را یادداشت کنید.
4. فهرست مصرف‌کنندگان فعلی MarketIntelligence را ثبت کنید.

Migration

```
dotnet ef migrations add MarketData_Phase1_NewTables -p MarketAnalyzer.Persistence
-s MarketAnalyzer.Api
```

فایل Migration تولیدشده را باز کنید. باید فقط ۹ جدول جدید در schema `marketintelligence` بسازد:
`MarketDefinitions` ، `MarketSymbols` ، `HistoricalCandles` ، `CurrentSymbolStates` ،

`InvestorTradingActivities`، `CorporateActions`، `CurrentOrderBookLevels`، `FundNavPoints` و `MarketSettings`. اگر چیز دیگری در آن بود، آن را اجرا نکنید و به من بگویید.

دو نکته

- موجود `Timeframe` عضوی به نام `Mountly` دارد (غلط املایی). به آن دست نزدم، چون تغییرش روی داده و enum مصرف‌کنندگان اثر می‌گذارد. در فاز پاک‌سازی تصمیم می‌گیریم.
- باگ نگاشت فیلدهای `TseBestLimit` قدیمی در مدل جدید با فیلدهای `Price`، `Volume` و `OrderCount` رفع شده است.

دو سؤال برای فاز ۲

- آیا اسم پروژهٔ `Collectors` به `Providers` عوض شود؟ هنوز جواب نداده‌اید و فاز ۴ به آن بستگی دارد.
- اگر Migration مشکلی نداشت و build سبز شد، فاز ۲ (Repositoryهای Persistence با Upsert اتمی) را شروع می‌کنم.

کل قسمت marketintelligence رو پاک کنم این رو کپی کنم؟

نه، پاک نکنید. zip فقط فایل‌های جدید دارد و روی مسیر پروژه کپی می‌شود (Merge، بدون جایگزینی چیزی).

چرا پاک نکنید

- کدهای قدیمی (Scheduler، Seeds، Infrastructure، Collectors و مصرف‌کنندگان) تا فاز ۶–۷ باید کار کنند و build سالم بماند.
- فایل‌های ما با فایل‌های قدیمی هم‌نام نیستند. نام جدول‌ها هم فرق دارد (`MarketDefinitions` به‌جای `Markets`) و تداخلی پیش نمی‌آید.
- پاک‌سازی در فاز ۷ با یک اسکریپت PowerShell انجام می‌شود، وقتی همه‌چیز جایگزین شده باشد.

مراحل

1. branch جدید بسازید و از دیتابیس backup بگیرید.
2. zip را روی ریشهٔ `backend` پوشهٔ solution را باز کنید و کپی کنید. اگر پرسید «Skip یا Replace»، نباید هیچ فایل هم‌نامی وجود داشته باشد. اگر وجود داشت، اسمش را به من بگویید.

3. `dotnet build` بزنید.
4. دستور Migration را اجرا کنید و فایلش را چک کنید که فقط ۹ جدول جدید بسازد.

اگر build خطا داد، متن خطا را بفرستید.

اوکی میخوام یه branch بزنم ولی روی dev باشم همچنان

یعنی branch جدید بسازی ولی روی dev بمونی؟ دو حالت داره:

۱. branch بساز ولی checkout نکن (روی dev بمان)

```
git branch feature/market-intelligence-redesign
```

این فقط branch رو می‌سازه و روی dev می‌مونی. ولی اگه فایل‌های zip رو الان کپی کنی، توی dev کامیت می‌شن نه branch جدید.
پس این حالت فقط برای وقتیه که بعداً می‌خوای بری روش.

۲. branch بساز و بعد روش کار کن (پیشنهادی)

```
git switch dev
git pull
git switch -c feature/market-intelligence-redesign
```

وقتی کارت تموم شد یا خواستی به dev برگردی: `git switch dev` .

اگه منظورت اینه که کدهای جدید توی dev هم باشن، این کار رو نکن. تا فاز ۷ کد نیمه‌کاره‌ست و dev نباید شکسته بشه. روی branch کار کن و آخر با PR به dev merge کن.

اگه کار نیمه‌کاره‌ای روی dev داری که هنوز commit نشده، قبل از `switch` اون رو `git stash` کن یا commit کن.

من برنچ رو میخوام که کد رو تا اینجا داشته باشم پس برنچ بزنم بریم جلو آخری merge کنیم؟

بله، درسته. روی branch کار کن و آخر merge کن.

```
git switch dev
git pull
git switch -c feature/market-intelligence-redesign
```

پیشنهادها

- بعد از هر فاز که build سبز شد commit کن (مثلاً `Phase 1: contracts and model`). اگر فازی خراب شد، می‌توانی به commit قبلی برگردی.
- اگر dev در این مدت تغییر کرد، گاهی `git merge dev` را داخل branch بزن تا آخر کار conflict بزرگ نشود.
- Migration را فقط روی branch و روی یک دیتابیس تست اجرا کن، نه روی دیتابیس مشترک dev. چون در فاز ۷ جدول‌های قدیمی drop می‌شوند، تا آن موقع به dev merge نکن.
- merge نهایی را با PR انجام بده، چون تغییرات بزرگ است و بازبینی می‌خواهد.

وقتی branch ساخته شد و zip را کپی کردی، build را بزن و نتیجه را بفرست.

ساخته شد کپی کردم build زدم اوکی بود

Ran 9 commands, shared files

فاز ۲ را تحویل دادم: `MarketIntelligence-Phase2.zip` . این کد هم کامپایل نشده است، چون `dotnet` در محیط من نیست.

قبل از فاز ۲ یک چیز را هنوز نگفتی: Migration فاز ۱ را اجرا کردی و فقط ۹ جدول جدید ساخت؟ اگر نه، اول آن را چک کن.

محتوا (۲۶ فایل)

- Domain (`Interface/MarketData/`): ۹ interface برای Store و دو نوع نتیجه (`UpsertStats` و `SymbolSyncResult`).
- Persistence (`Stores/`): پیاده‌سازی EF هر Store، `UniqueViolation` و `AddMarketDataStores()` برای ثبت در DI.
- **فایل `.csproj` Persistence:** به `ProjectReference` این تنها فایل موجودی است که عوض می‌شود. فقط یک را شخصی‌سازی کرده‌ای، فقط همان یک خط را csproj اضافه شده تا `IDbContext` در دسترس باشد. اگر Abstractions دستی اضافه کن.

راه‌اندازی: `services.AddMarketDataStores()` را جایی صدا بزن که Infrastructure ثبت می‌شود (فعلاً کنار `ServiceCollectionExtensions` قدیمی). `TimeProvider` هم همان‌جا ثبت می‌شود.

تصمیم‌های فنی که باید بدانی

- **کندل:** فقط بازهٔ ورودی خوانده می‌شود، و EF فقط برای ستون‌هایی که واقعاً عوض شده‌اند UPDATE می‌فرستد.
- **Race condition:** یک‌بار با Store، ایندکس یکتا نگهبان نهایی است. اگر دو نویسندهٔ همزمان یک رکورد جدید یکسان بسازند باشد که همزمان اجرا نمی‌شود job وضعیت تازه دوباره تلاش می‌کند. نویسندهٔ اصلی هر نماد باید.
- **جایگزینی اتمی:** `ReplaceAllAsync` حذف و درج را در یک تراکنش انجام می‌دهد. ورودی خالی را رد می‌کند تا شکست منبع هیچ‌وقت سری را پاک نکند.
- **صف سفارش‌ها:** همهٔ نمادهای یک batch در یک تراکنش جایگزین می‌شوند و خواننده هرگز ترکیب قدیم و جدید نمی‌بیند.
- **وضعیت جاری:** وضعیتی که زمان منبعش قدیمی‌تر از مقدار ذخیره‌شده باشد، جایگزین نمی‌شود.
- **تنظیمات:** `InsertIfMissing` ویرایش اپراتور را هرگز بازنویسی نمی‌کند.

دو هشدار

- `SaveChangesAsync` در `FundDbContext` وقتی برنامه در حالت قفل (`_applicationLock`) باشد خطا می‌دهد. اگر هنگام قفل، ingestion اجرا شود شکست می‌خورد. در فاز ۳ برای این حالت رفتار صریح می‌گذارم: job رد شود و دوباره تلاش کند.
- بعد از هر نوشتن، `ClearChangeTracker` صدا زده می‌شود. هر کدی که در همان scope موجودیت tracked دارد، آن را از دست می‌دهد. مصرف‌کننده‌ها باید با `AsNoTracking` بخوانند، یا هر job یک scope جدا داشته باشد.

فاز ۳ (هستهٔ ingestion): اعتبارسنجی، جداسازی کندل ناتمام، تشخیص drift، اتمی‌سازی refetch و ساخت هفتگی/ماهانه با تقویم. با build سبز فاز ۲ شروع می‌کنم. فایل‌ها را کپی کن، build بزن و نتیجه را بگو.

کنم؟ replace

بله، فقط یک فایل جایگزین می‌شود: `MarketAnalyzer.MarketIntelligence.Persistence.csproj`. بقیهٔ فایل‌ها جدیدند و هیچ‌کدام هم‌نام فایل موجود نیست.

اگر csproj را شخصی‌سازی نکرده‌ای، Replace بزن. اگر Git داری، بعد از کپی `git diff` روی همان csproj بگیر. فقط باید یک `ProjectReference` به `MarketAnalyzer.Abstractions` اضافه شده باشد، بدون هیچ تغییر دیگری.

بعدش `dotnet build` بزن و نتیجه را بفرست. اگر سبز بود، فاز ۳ را شروع می‌کنم.

بیلد اوکی بود

Ran 4 commands, shared files

فاز ۳ را تحویل دادم: `MarketIntelligence-Phase3.zip`. هنوز کامپایل نشده است، پس بعد از کپی `dotnet build` بزن.

دو فایل از فاز ۲ جایگزین می‌شوند: `IHistoricalCandleStore.cs` و `EfHistoricalCandleStore.cs`. فقط متد `ReplaceSeriesAsync` به آن‌ها اضافه شده است. بقیهٔ ۲۴ فایل جدیدند و زیر `Application/MarketData/Ingestion/` می‌روند.

هسته (`HistoricalCandleIngestor`)

- اولین بار تاریخچهٔ کامل را می‌گیرد. بعد از آن، هر بار فقط ۱۰۰ کندل آخر را می‌گیرد.
- اگر قیمت‌های قدیمی همه با یک ضریب ثابت عوض شده باشند (افزایش سرمایه یا سود)، همان نماد را کامل دوباره می‌گیرد. روزانه، هفتگی و ماهانه در یک تراکنش جایگزین می‌شوند. رویداد هم با `DetectedFromDrift` در `CorporateActions` ثبت می‌شود.
- جایگزینی فقط با نتیجهٔ Success انجام می‌شود. شکست، پاسخ خالی و پاسخ ناقص هرگز چیزی را پاک نمی‌کنند. اگر تاریخچهٔ تازه از پنجرهٔ ذخیره‌شده کوتاه‌تر باشد، یا بیش از ۲۰٪ کندل‌ها نامعتبر باشند، جایگزینی انجام نمی‌شود.

تصمیم‌های من که باید تأییدشان کنی

1. هفتهٔ بازار ایران از **شنبه** شروع می‌شود و ماهش **ماه شمسی** است. بازارهای دیگر دوشنبه و ماه میلادی دارند. اگر برای ایران ماه میلادی می‌خواهی، بگو.
2. کندل «امروز» تا نیمه‌شب محلی ناتمام حساب می‌شود، حتی بعد از بسته شدن بورس. بعد از نیمه‌شب وارد جدول تاریخی می‌شود. کندل ناتمام در `OpenCandle` نتیجه برمی‌گردد و ذخیرهٔ تاریخی نمی‌شود.
3. نوع رویداد drift را `Other` می‌گذارم، چون از روی قیمت معلوم نیست افزایش سرمایه بوده یا سود نقدی. تعریف ضریب: قیمت جدید ÷ قیمت ذخیره‌شده. فرمول Raw را در فاز ۴ با داده‌های واقعی چک می‌کنیم.
4. تلرانس‌ها در `IngestionOptions` قابل تنظیم‌اند. مثلاً تلرانس ۰٫۲٪ برای گردشدن قیمت‌ها و پنجرهٔ ۱۰۰ کندل. اگر قیمت‌های تعدیل‌شدهٔ نهایت‌نگر بیشتر گرد می‌شوند، با داده‌های واقعی تنظیمشان می‌کنیم.

نیاز دارد در Program/DI صدا بزنی

```
services.AddMarketDataStores();
services.AddMarketDataIngestion();
services.Configure<IngestionOptions>
(configuration.GetSection(IngestionOptions.SectionName)); // اختیاری
```

محدودیت: اگر برنامه در حالت قفل (`_applicationLock`) باشد، ذخیره شکست می‌خورد و نتیجه `Failed` برمی‌گردد. تلاش دوباره را در فاز ۵ (scheduler) پیاده می‌کنم.

فاز ۴ (Providerها): ۹ منبع قدیمی را به Provider جدید تبدیل می‌کنم. یک سؤال دارم: اسم پروژهٔ `Collectors` به `Providers` عوض شود؟ هنوز جواب نداده‌ای. اگر بله، پروژهٔ جدید `MarketAnalyzer.MarketIntelligence.Providers` می‌سازم و کد قدیمی را تا فاز ۷ نگه می‌دارم. اگر نه، همان‌جا کار می‌کنم. می‌خواهی با کدام منبع شروع کنم؟ پیشنهادم Nahayatnegar (بورس) است، چون پیچیده‌ترین است و بقیه از آن ساده‌ترند.

1- همه تاریخ ها یکی باشه ترجیحا که بهتره میلادی باشه
2-کلا کندلی که تغییر میکنه نباید توhistory بشینه چک میکنیم اگه فلان کندل فلان تاریخ نبود میشونیمش
ولی اگر کندل داره تغییر میکنه میره تو روزانه نمیدونم چطوری هندل اش میکنی

3-اوکی
4-اوکی

اسم ها رو کلا هرچی به نظرت درسته بزار اینجا هم به نظرم provider اوکی هست

الان بورس ایران رو داریم از Nahayatnegar میگیریم ولی ممکنه بخواهیم جایگزین اش کنم ویه جای دیگه باشه در این صورت دیگه نباید Nahayatnegar رو بگیریم
در کل برای هر بازار باید از یک منبع بگیریم و انتخاب کنبم این اوکیه؟

Ran 2 commands, shared files

فاز ۳ را اصلاح و دوباره فرستادم. اگر نسخهٔ قبلی را کپی کرده‌ای، این را روی آن Replace کن (همان ۲۶ فایل، با همان مسیرها).

۱. تاریخ میلادی برای همه
تقویم شمسی و هفتهٔ شنبه‌ای را حذف کردم. حالا همهٔ بازارها هفته را از **دوشنبه** و ماه را **میلادی** حساب می‌کنند. یک نکته: هفتهٔ معاملاتی بورس ایران شنبه تا چهارشنبه است. با شروع از دوشنبه، کندل هفتگی ایران «شنبه و یکشنبه» را به هفتهٔ قبل می‌دهد و بقیه را به هفتهٔ جدید. برای مقایسه بین بازارها همین یک‌دستی را ترجیح دادم. اگر کندل هفتگی ایران برای تحلیل مهم است، بگو تا شروع هفته را برای هر بازار قابل تنظیم کنم.

۲. کندلی که تغییر می‌کند
سه حالت جدا هستند:

- **کندل امروز (ناتمام):** هرگز وارد جدول تاریخی نمی‌شود. فقط وقتی بازه کاملاً بسته شد ذخیره می‌شود. تا آن موقع داده‌اش در `CurrentSymbolStates` است.
- **کندل بسته که منبع بعداً اصلاحش می‌کند:** هر دریافت دوره‌ای ۱۰۰ کندل آخر را دوباره می‌گیرد و مقدار را overwrite می‌کند. هفتگی و ماهانه هم از همان نقطه دوباره ساخته می‌شوند.
- **کندلی که منبع دیگر نمی‌دهد (حالت پیشنهادی تو):** اضافه کردم. اگر کندل ذخیره‌شده داخل بازهٔ پاسخ باشد ولی در پاسخ نباشد، حذف می‌شود. فقط با پاسخ کامل Success. اگر بیش از ۲۰٪ پنجره حذف‌شدنی باشد، حذف نمی‌کند و فقط warning می‌دهد، چون احتمالاً پاسخ ناقص است. اگر کندل پایه‌ای حذف شود، بازه‌های هفتگی و ماهانهٔ بعد از آن از صفر ساخته می‌شوند.

۳ و ۴. تأیید.

منبع فعال هر بازار: بله، این درست است. پیاده‌سازی‌اش این‌طور است:

- در `MarketSettings` برای هر بازار یک کلید `ActiveProviderKey` هست (مثلاً `tse` → `nahayatnegar`). فقط Provider فعال job می‌گیرد؛ بقیه کد هستند ولی هیچ‌وقت صدا زده نمی‌شوند.

- بازار جدید یا منبع جایگزین بورس = یک کلاس Provider جدید و عوض کردن آن تنظیم. نیازی به seed یا enum نیست.
- **وقتی منبع عوض می‌شود:** تصمیم این است که کندل‌های آن بازار پاک شوند و از منبع جدید کامل دوباره گرفته شوند، چون تعدیل و گردکردن قیمت هر منبع فرق دارد و ترکیب دو منبع در یک سری، drift کاذب می‌سازد. نمادها با `SymbolKey` استاندارد یکی می‌مانند. نمادی که منبع جدید ندارد غیرفعال می‌شود و حذف نمی‌شود. تغییر منبع در API یک تأیید جدا می‌خواهد، چون پاک‌کردن را به دنبال دارد. اگر مخالفی، بگو.

نام‌ها: پروژهٔ جدید `MarketAnalyzer.MarketIntelligence.Providers` می‌شود و Collectors قدیمی تا فاز ۷ می‌ماند.

فاز ۴: با Nahayatnegar شروع می‌کنم. برایش لازم دارم فایل‌های قدیمی `NahayatnegarCandleSourceBase` و کلاس‌های TseIndex ،Tse و تنظیمات URL را ببینم. آن‌ها در zip قبلی هستند و من خودم می‌خوانم، پس چیزی لازم نیست بفرستی. فقط بگو: لیست نمادهای بورس را همین الان از کجا می‌گیری؟ (`TseTmcClient` یا Nahayatnegar؟)

بیلد اوکی بود

Ran 11 commands

This response didn’t load.

بیلد اوکی بود

Ran a command, shared files

فاز ۴-الف را تحویل دادم: `MarketIntelligence-Phase4a.zip` (۱۶ فایل). هنوز کامپایل نشده است و فقط با بازبینی دستی چک شده.

یک اصلاح در تصمیم قبلی

در کد قدیمی دیدم که در بورس، فهرست نمادها از TSETMC می‌آید و کندل از نهایت‌نگر. پس «یک منبع برای هر بازار» کافی نیست. منبع فعال را برای هر **(بازار، نوع داده)** انتخاب می‌کنیم. مثلاً بازار `tse`: فهرست نماد = tsetmc، کندل = nahayatnegar. هر کدام را جدا می‌توانی عوض کنی. برای هر (بازار، نوع داده) هنوز فقط یک منبع فعال است و fallback خودکار نداریم. اگر مخالفی، بگو.

محتوا

- Application (۷ **فایل جایگزین و ۱ فایل جدید**): انتخاب منبع فعال از `MarketSettings` با کلید `Provider:HistoricalCandles` scope در Provider بازار، و خواندن تنظیمات از دیتابیس. `MarketDataIngestionRegistration.cs` جایگزین نسخهٔ قبلی می‌شود.
- **پروژهٔ جدید** `MarketAnalyzer.MarketIntelligence.Providers`:
  - کلاینت نهایت‌نگر،
  - دو Provider: `tse` (سهام، AlreadyAdjusted) و `tse-index` (شاخص، بدون تعدیل).
  - آدرس کندل فعلاً از `url_settings.json` قدیمی خوانده می‌شود و Bootstrap فاز ۵ آن را یک‌بار داخل دیتابیس کپی می‌کند.

کارهای دستی لازم (یک‌بار)

```
dotnet sln MarketAnalyzer.sln add
MarketIntelligence/src/MarketAnalyzer.MarketIntelligence.Providers/MarketAnalyzer.
MarketIntelligence.Providers.csproj
```

بعد در Api یک `ProjectReference` به این پروژه بده و در `Program.cs` این خط را اضافه کن:

```
builder.Services.AddMarketDataProviders();
```

تفاوت‌های مهم با کد قدیمی

- قبلاً هر خطا (شبکه، JSON خراب) به «لیست خالی» تبدیل می‌شد. الان `Failed` برمی‌گردد و هیچ داده‌ای پاک نمی‌شود. `no_data` می‌شود `SuccessEmpty`.
- نماد در URL با `Uri.EscapeDataString` encode می‌شود (جلوگیری از تزریق پارامتر).
- رفتار قدیمی حفظ شده: به نماد عدد `3` (تعدیل‌شده) اضافه می‌شود. نمادی که با `IRR` شروع شود `rights` است و بقیه `stock`.
- نماد ارسالی به نهایت‌نگر همان `InstrumentId` است (مثل `IRO1FOLD0001`)، پس کلید نماد بورس در کاتالوگ جدید `InstrumentId` خواهد بود.

دو چیز که نمی‌توانم از اینجا چک کنم (شبکه بسته است)

1. فرمت پاسخ نهایت‌نگر را از روی کد قدیمی فرض کرده‌ام (آرایه‌های `t,o,h,l,c,v`). اگر پاسخ واقعی فرق داشته باشد، نتیجه `Failed` می‌شود و پیام خطا در لاگ می‌آید.
2. برای شاخص‌ها، کد قدیمی هم همین `3` را به نماد اضافه می‌کرد. مطمئن نیستم برای شاخص درست باشد. اگر کندل شاخص خالی آمد، بگو.

قدم بعدی (۴-ب): فهرست نماد و وضعیت جاری بورس از TSETMC. در کد قدیمی دیدم `GetAllIndexAsync` یک تاریخ ثابت (`20260608`) دارد و یک حلقهٔ بی‌دلیل چهاربار تکرار. آن را در Provider جدید درست می‌کنم. برای TSETMC، `User` و `Password` وب‌سرویس باید از User Secrets بیاید. الان در `UrlSettings` هستند؟ اگر بله، آدرس فایل را بگو. بعد از آن، بقیه منبع‌ها (کریپتو، ماکرو، IME، مسکن، صندوق‌ها) یکی‌یکی می‌آیند.

در مورد دوتا سوالت اینکه کد قدیمی کار میکرد دیتا رو میگرفت و اوکی بود فقط اون داستان معماری ها و مواردی که گفتم مشکل داشت
در مورد اضافه کرد 3 به نماد هم به خاطر اینه که اونجا چند مدل دیتا میده 3 دیتای تعدیل شده رو میده

Read memory, ran a command

This response didn’t load.

```
PS D:\Total Solution\My-Fund-git> powershell -ExecutionPolicy Bypass -File .\gm-run.ps1 -From 2024-08-15 -To 2025-06-30 -Tag Y1403 -BackupFile "D:\DataBase\Backup\db_FundAudit_GM_PRISTINE_ReadyForShadow_v3.bak" -SkipAssetsRun -SkipBourseRun -NoBuild
[gm-run] range 2024-08-15..2025-06-30  tag=Y1403  db=db_FundAudit_GM
[gm-run] restoring db_FundAudit_GM from
```

```
D:\DataBase\Backup\db_FundAudit_GM_PRISTINE_ReadyForShadow_v3.bak ...
[gm-run] applying GM fixes ...
D:\Total Solution\My-Fund-git\gm-run.ps1 : Exception calling "ExecuteNonQuery"
with "0" argument(s): "Execution
Timeout Expired.  The timeout period elapsed prior to completion of the operation
or the server is not responding.
Operation cancelled by user.
The statement has been terminated."
    + CategoryInfo          : NotSpecified: (:) [gm-run.ps1], MethodInvocationException
    + FullyQualifiedErrorId : SqlException,gm-run.ps1
```

Show more

Used 2 tools, ran a command

`Execution Timeout Expired` در مرحلهٔ `applying GM fixes` یعنی یکی از دو `UPDATE` (روی `accounting.FiscalYears` یا `bourse.Instruments`) بیش از ۳۰ ثانیه منتظر ماند. این دو UPDATE روی جدول‌های کوچک‌اند و باید فوری تمام شوند. پس تقریباً مطمئنم **چیزی روی** `db_FundAudit_GM` **قفل نگه داشته** و خود UPDATE کند نیست. من اسکریپت `gm-run.ps1` را ندارم، پس این تشخیص را از روی خطا می‌گویم.

محتمل‌ترین علت‌ها، به ترتیب:

1. اجرای قبلی (مثلاً Dec24p که در runbook «در حال اجرا» بود) نیمه‌کاره بسته شده و پروسهٔ `dotnet` هنوز زنده است، یا transaction بازی روی همین DB دارد.
2. یک پنجره SSMS روی `db_FundAudit_GM` که transaction بی‌commit دارد (حتی یک `SELECT` داخل `BEGIN TRAN`).
3. Api یا برنامهٔ دیگری که به همین DB وصل است.

۱. پروسه‌های باقی‌مانده را ببین (PowerShell)

```powershell
```

```
Get-Process dotnet, gm-run -ErrorAction SilentlyContinue | Select Id, ProcessName, S
```

اگر `dotnet` قدیمی از اجرای قبلی هست، ببندش: `Stop-Process -Id <Id>` .

۲. در SSMS ببین چه چیزی بلاک می‌کند

```sql
SELECT r.session_id, r.blocking_session_id, r.wait_type, r.wait_time,
       s.login_name, s.host_name, s.program_name, s.status,
       DB_NAME(r.database_id) AS db, t.text
FROM sys.dm_exec_sessions s
LEFT JOIN sys.dm_exec_requests r ON r.session_id = s.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE s.database_id = DB_ID('db_FundAudit_GM') AND s.session_id <> @@SPID;

DBCC OPENTRAN('db_FundAudit_GM');
```

- هر ردیفی که `blocking_session_id` دارد، قربانی است. ردیف بلاک‌کننده را پیدا کن.
- اگر `DBCC OPENTRAN` یک transaction قدیمی نشان داد، با `KILL <session_id>` ببندش (فقط session مربوط به GM، نه چیز دیگر).

۳. اگر چیزی پیدا نشد

احتمالاً DB بعد از restore هنوز در حال recovery است یا auto-close/auto-shrink روشن است. بعد از restore یک بار این را بزن و دوباره اجرا کن:

```sql
ALTER DATABASE db_FundAudit_GM SET MULTI_USER;
SELECT name, state_desc, user_access_desc FROM sys.databases WHERE name='db_FundAudi
```

`state_desc` باید `ONLINE` باشد.

۴. اصلاح پایدار در `gm-run.ps1`
تایم‌اوت پیش‌فرض ۳۰ ثانیه است. برای مرحلهٔ fixes (و restore) `CommandTimeout` را صریح بگذار، مثلاً `$cmd.CommandTimeout = 300`. بعد از restore هم اسکریپت باید قبل از fixes منتظر `ONLINE` بماند و اگر session دیگری روی DB بود، با `ALTER DATABASE ... SET SINGLE_USER WITH ROLLBACK IMMEDIATE` آن را بیندازد و بعد `MULTI_USER` کند. اگر متن اسکریپت را بفرستی، دقیقاً همان‌جا اصلاحش می‌کنم.

یک نکتهٔ جدا: با `-SkipAssetsRun -SkipBourseRun` فقط restore و fixes و مقایسه اجرا می‌شود و هیچ run واقعی انجام نمی‌شود. اگر هدفت ادامهٔ اجرای Y1403 است، این دو سوییچ را برندار، وگرنه خروجی معنایی ندارد.

بعد از رفع بلاک، دوباره همان دستور را بزن. restore دوباره انجام می‌شود و چیزی خراب نمی‌ماند. نتیجهٔ کوئری مرحلهٔ ۲ را بفرست تا اگر چیز غیرمنتظره‌ای بود کمک کنم. بعد از آن، فاز ۴-ب (TSETMC) را ادامه می‌دهم.

```
PS D:\Total Solution\My-Fund-git> Get-Process dotnet, gm-run -ErrorAction
SilentlyContinue | Select Id, ProcessName, StartTime

  Id ProcessName StartTime
  -- ----------- ---------
 3744 dotnet     10/26/2026 9:52:35 AM
13636 dotnet     10/26/2026 9:52:35 AM
20900 dotnet     10/26/2026 9:52:35 AM
21700 dotnet     10/26/2026 9:52:35 AM
23544 dotnet     10/26/2026 9:52:35 AM
25952 dotnet     10/26/2026 9:52:35 AM
35400 dotnet     10/26/2026 9:52:35 AM
```

این کوئری خالی بود

```
SELECT r.session_id, r.blocking_session_id, r.wait_type, r.wait_time,
    s.login_name, s.host_name, s.program_name, s.status,
    DB_NAME(r.database_id) AS db, t.text
FROM sys.dm_exec_sessions s
LEFT JOIN sys.dm_exec_requests r ON r.session_id = s.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE s.database_id = DB_ID('db_FundAudit_GM') AND s.session_id <>
@@SPID;

DBCC OPENTRAN('db_FundAudit_GM');
```

و اونو زدم
name state_desc user_access_desc
db_FundAudit_GM ONLINE MULTI_USER

Show more

کوئری خالی بود چون وقتی اجرایش کردی اسکریپت قبلاً شکست خورده و session خودش بسته شده بود. پس چیزی از آن لحظه نمی‌بینیم.
DB الان سالم است (`ONLINE` و `MULTI_USER`).

هفت پروسهٔ `dotnet` با زمان شروع یکسان (۹:۵۲:۳۵) احتمالاً به هم ربط دارند و از یک parent شروع شده‌اند، مثل Visual Studio یا build server. بعید است قفل DB را نگه داشته باشند، ولی چک کن:

```powershell
Get-CimInstance Win32_Process -Filter "Name='dotnet.exe'" | Select ProcessId, Parent
```

اگر `CommandLine` یکی‌شان شامل `--control-run` / `Api` / `MarketAnalyzer` بود، به DB وصل است. بقیه (`build-server`, `MSBuild`, `VBCSCompiler`) بی‌ضررند.

قدم اصلی: ببین در لحظهٔ گیر کردن چه چیزی اجرا می‌شود
۱. در SSMS این کوئری را آماده نگه دار (بدون `database_id` که session اسکریپت را از دست ندهی):

```sql
SELECT r.session_id, r.blocking_session_id, r.wait_type, r.wait_time, r.status,
       DB_NAME(r.database_id) AS db, r.command,
       SUBSTRING(t.text, r.statement_start_offset/2 + 1,
         (CASE r.statement_end_offset WHEN -1 THEN DATALENGTH(t.text) ELSE r.stateme
FROM sys.dm_exec_requests r
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id > 50 AND r.session_id <> @@SPID;
```

۲. اسکریپت را دوباره اجرا کن. وقتی «applying GM fixes ...» چاپ شد، **فوراً** (در ۳۰ ثانیه) کوئری را بزن.
۳. خروجی `blocking_session_id`، `wait_type` و `running_statement` را بفرست.

تفسیر نتیجه

- `wait_type = LCK_M_*` و `blocking_session_id` پر: بلاک واقعی. باید معلوم شود چه کسی با `DBCC INPUTBUF(<blocking_id>)` ببین چیست.
- `running_statement` یک دستور سنگین است (مثلاً rebuild، `UPDATE STATISTICS` یا کپی بزرگ): پس «fixes» فقط آن دو UPDATE نیست. باید متن `gm-run.ps1` / `FixesSql` را ببینم.
- `wait_type = PAGEIOLATCH_*`: I/O کند بعد از restore (دیسک). ممکن است فقط CommandTimeout را بالا ببری و حل شود.

تست دستی سریع (قبل از اجرای دوباره، در SSMS روی `db_FundAudit_GM`):

```sql
SET LOCK_TIMEOUT 5000;  SET STATISTICS TIME ON;
BEGIN TRAN;
UPDATE bourse.Instruments SET IssuingPurposeId = 1 WHERE Id = N'IRSARPAZ0161';
UPDATE accounting.FiscalYears SET Status = 1 WHERE Id = 12;
ROLLBACK;
```

اگر یکی‌شان خطای `Lock request time out` داد یا چند ثانیه طول کشید، همان جدول مشکل دارد. در این حالت هم تعداد trigger را ببین:

```sql
SELECT OBJECT_SCHEMA_NAME(parent_id) s, OBJECT_NAME(parent_id) t, name FROM sys.trig
WHERE parent_id IN (OBJECT_ID('bourse.Instruments'), OBJECT_ID('accounting.FiscalYea
```

ساده را کند کند UPDATE سنگین روی این دو جدول می‌تواند audit یا Trigger.

نتیجهٔ مرحلهٔ ۲ و تست دستی را بفرست. اگر متن `gm-run.ps1` را هم بفرستی (یا فقط بخش fixes و تنظیمات اتصال)، همان‌جا `CommandTimeout` و انتظار برای ONLINE را اصلاح می‌کنم.

```
SET LOCK_TIMEOUT 5000;  SET STATISTICS TIME ON;
BEGIN TRAN;
UPDATE bourse.Instruments SET IssuingPurposeId = 1 WHERE Id =
N'IRSARPAZ0161';
UPDATE accounting.FiscalYears SET Status = 1 WHERE Id = 12;
ROLLBACK;
```

خطایی نداد

قبل از اجرای دوباره

```
SELECT r.session_id, r.blocking_session_id, r.wait_type, r.wait_time, r.status,
    DB_NAME(r.database_id) AS db, r.command,
    SUBSTRING(t.text, r.statement_start_offset/2 + 1,
        (CASE r.statement_end_offset WHEN -1 THEN DATALENGTH(t.text) ELSE
            r.statement_end_offset END - r.statement_start_offset)/2 + 1) AS
            running_statement
FROM sys.dm_exec_requests r
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id > 50 AND r.session_id <> @@SPID;
```

این رو زدم

که اینو داد

```
session_id  blocking_session_id  wait_type  wait_time  status  db  command
running_statement
53  0  NULL  0  sleeping  master  TASK MANAGER  NULL
54  0  NULL  0  sleeping  master  TASK MANAGER  NULL
55  0  HADR_NOTIFICATION_DEQUEUE  161871046  background  master
HADR_AR_MGR_NOTIFICATION_WORKER  NULL
59  0  NULL  0  sleeping  master  TASK MANAGER  NULL
60  0  NULL  0  sleeping  master  TASK MANAGER  NULL
61  0  NULL  0  sleeping  master  TASK MANAGER  NULL
66  0  NULL  0  sleeping  master  TASK MANAGER  NULL
67  0  NULL  0  sleeping  master  TASK MANAGER  NULL
68  0  NULL  0  sleeping  master  TASK MANAGER  NULL
```

69	0	NULL	0	sleeping	master	TASK MANAGER	NULL
70	0	NULL	0	sleeping	master	TASK MANAGER	NULL
71	0	NULL	0	sleeping	master	TASK MANAGER	NULL
72	0	NULL	0	sleeping	master	TASK MANAGER	NULL
73	0	NULL	0	sleeping	master	TASK MANAGER	NULL
74	0	NULL	0	sleeping	master	TASK MANAGER	NULL

Show more

تست دستی یعنی دو UPDATE فوری انجام می‌شوند و DB مشکلی ندارد. پس علت باید چیز دیگری باشد، که فقط وقتی اسکریپت واقعاً گیر کرده معلوم می‌شود. خروجی کوئری هم فقط sessionهای سیستمی بی‌کار بود.

۱. **فیلتر کوئری را عوض کن** تا نویز سیستمی حذف شود و session اسکریپت دیده شود:

```sql
SELECT r.session_id, r.blocking_session_id, r.wait_type, r.wait_time, r.status,
       DB_NAME(r.database_id) AS db, r.command, s.program_name, s.host_name,
       SUBSTRING(t.text, r.statement_start_offset/2 + 1,
         (CASE r.statement_end_offset WHEN -1 THEN DATALENGTH(t.text) ELSE r.stateme
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON s.session_id = r.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE s.is_user_process = 1 AND r.session_id <> @@SPID;
```

۲. **اسکریپت را دوباره اجرا کن** و بعد از چاپ «applying GM fixes ...» همین کوئری را چند بار پشت‌سر هم بزن (۳۰ ثانیه فرصت داری). اگر هیچ ردیفی نبود، یعنی دستور واقعاً به SQL نرسیده و مشکل سمت اسکریپت است.

۳. **متن بخش fixes اسکریپت را بفرست.** در `gm-run.ps1` دنبال خطی بگرد که بعد از `applying GM fixes` است (محتوای `-FixesSql` یا همان دو UPDATE). فقط همان چند خط و خط ساخت connection/command کافی است. حدس‌هایم:

- اسکریپت fixes چیزی بیشتر از دو UPDATE دارد (مثلاً `gm-05`، snapshot یا rebuild) که در تست دستی من نبود.
- `ExecuteNonQuery` همهٔ فایل را با یک `CommandTimeout` پیش‌فرض ۳۰ ثانیه اجرا می‌کند و فایل `GO` دارد. `GO` دستور T-SQL نیست و `ExecuteNonQuery` آن را نمی‌فهمد، یا همهٔ batchها را به‌صورت یک دستور بزرگ می‌فرستد.

- connection بعد از restore به `master` یا به DB قدیمی وصل می‌ماند و روی pool متصل به DB در حال restore گیر می‌کند.

با دیدن متن، `CommandTimeout` را صریح و بزرگ می‌کنم و اگر `GO` هست batchها را جدا اجرا می‌کنم.

اشتباه شد این کد ها برای اینجا نبود برو فاز بعدی

Ran 9 commands, shared files

فاز ۴-ب را تحویل دادم: `MarketIntelligence-Phase4b.zip`. کل پوشهٔ پروژهٔ Providers است (۲۵ فایل). روی نسخهٔ ۴-الف Replace کن، چون csproj و `ProvidersRegistration.cs` هم عوض شده‌اند. کامپایل نشده است، پس `dotnet build` بزن.

Provider(`tsetmc` منبع) `tse` های جدید بازار

- **فهرست نماد:** کلید = `InstrumentId`، `ExternalId` = `InsCode`. در `IsActive` همان قاعدهٔ قدیمی فیلتر کندل اعمال شده (IRO...0001، زیرصنعت ≠ 6812، Flow ≠ 4). بقیهٔ ابزارها ذخیره می‌شوند ولی غیرفعال‌اند. نماد کوتاه (مثل «فولاد») در `PersianName` است تا جست‌وجو روی آن کار کند.
- **وضعیت جاری:** از `TradeLastDay` و حد مجاز قیمت از `StaticThresholds`. زمان منبع تهران به UTC تبدیل می‌شود.
- **صف خرید/فروش:** باگ نگاشت فیلدهای قدیمی (قیمت و تعداد جابه‌جا) رفع شد. نمادی که صفش خالی شده با Snapshot خالی می‌آید تا صف قدیمی‌اش پاک شود.
- **خرید و فروش حقیقی/حقوقی:** با ۴ درخواست همزمان؛ فقط ۶۰ روز اخیر (قابل تنظیم، `0` = کل تاریخچه برای بار اول). اگر بعضی نمادها شکست بخورند نتیجه `Partial` است و فقط داده‌های سالم ذخیره می‌شوند. اگر همه شکست بخورند `Failed`.

فایل SOAP: `Reference.cs` را به پروژهٔ Providers کپی کردم و فقط namespaceاش را عوض کردم، تا با نسخهٔ Collectors تداخل نشود. در فاز ۷ نسخهٔ قدیمی با Collectors حذف می‌شود. بستن channel هم اضافه شد (قبلاً هرگز بسته نمی‌شد).

اصلاح‌های نسبت به کد قدیمی

- اگر یکی از ۴ بازار جریان (Flow) شکست بخورد، کل دریافت `Failed` می‌شود و دادهٔ ناقص ذخیره نمی‌شود.

- خواندن فیلدها با cast مستقیم نبود؛ فیلد گم‌شده فقط همان ردیف را رد می‌کند و کل لیست را خراب نمی‌کند.

مسئلهٔ امنیتی: نام کاربری و رمز وب‌سرویس TSETMC داخل `url_settings.json` است و آن فایل در Git می‌ماند. پیشنهاد می‌کنم:

1. هر دو را به User Secrets ببری (`MarketIntelligence:Providers:Tsetmc:User` و `...:Password`). Provider جدید اول از آنجا می‌خواند و فقط در صورت نبودن، از فایل قدیمی.
2. رمز را عوض کنی، چون در تاریخچهٔ Git باقی می‌ماند.
3. آدرس سرویس `http://` است، یعنی رمز بدون رمزنگاری می‌رود. اگر سرویس `https` دارد، در `MarketSettings` عوضش کن.

در `Program.cs` به‌جای قبلی بگذار:

```
builder.Services.AddMarketDataProviders(builder.Configuration);
```

قدم بعدی (۴-ج): بقیهٔ منبع‌ها یکی‌یکی: کریپتو (Wallex، Nobitex، Kucoin)، ماکرو (Tgju)، IME، مسکن (Kilid)، شاخص‌های بورس و صندوق‌ها (Fipiran). برای سرعت، از کریپتو شروع می‌کنم چون سه منبع یکسان UDF دارند و ساده‌ترند. یک سؤال دارم: Nobitex و Wallex هر دو بازار «کریپتو به تومان» را می‌دهند. همان یک بازار `crypto-toman` با دو منبع قابل‌تعویض باشد؟ (پیشنهاد من، مطابق قاعدهٔ «یک منبع فعال برای هر بازار»)

یه نکته که نماد ها دلاری هستن یا تومان یا ریال رو چطوری میتونیم هندل کنیم؟ الان کریپتویی ها دلاری هستن
بورس ریالی و کریپتو ها بعضی هاشون تومان هستن شاید چیز دیگه ای هم اضافه بشه
و بعد تو گزارش شاید بر اساس این ضریب بخوره

یه نکته که نماد ها دلاری هستن یا تومان یا ریال رو چطوری میتونیم هندل کنیم؟ الان کریپتویی ها دلاری هستن
بورس ریالی و کریپتو ها بعضی هاشون تومان هستن شاید چیز دیگه ای هم اضافه بشه
و بعد تو گزارش شاید بر اساس این ضریب بخوره

اینجا خطا هم دارم

System.AggregateException: 'Some services are not able to be constructed (Error while validating the service descriptor 'ServiceType: MarketAnalyzer.MarketIntelligence.Providers.TsetmcWebServiceClient Lifetime: Scoped ImplementationType: MarketAnalyzer.MarketIntelligence.Providers.TsetmcWebServiceClient': Unable to resolve service for type 'MarketAnalyzer.MarketIntelligence.Application.MarketData.Providers.IProviderSettings' while attempting to activate 'MarketAnalyzer.MarketIntelligence.Providers.TsetmcWebServiceClient'.) (Error while validating the service descriptor 'ServiceType: MarketAnalyzer.MarketIntelligence.Providers.TseSymbolDirectory Lifetime: Scoped ImplementationType: MarketAnalyzer.MarketIntelligence.Providers.TseSymbolDirectory': Unable to resolve service for type 'MarketAnalyzer.MarketIntelligence.Domain.IMarketDefinitionStore' while attempting to activate 'MarketAnalyzer.MarketIntelligence.Providers.TseSymbolDirectory'.) (Error while validating the service descriptor 'ServiceType: MarketAnalyzer.MarketIntelligence.Providers.TsetmcStockCatalogProvider Lifetime: Scoped ImplementationType: MarketAnalyzer.MarketIntelligence.Providers.TsetmcStockCatalogProvider': Unable to resolve service for type 'MarketAnalyzer.MarketIntelligence.Application.MarketData.Providers.IProviderSettings' while attempting to activate 'MarketAnalyzer.MarketIntelligence.Providers.TsetmcWebServiceClient'.) (Error while validating the service descriptor 'ServiceType: MarketAnalyzer.MarketIntelligence.Providers.TsetmcStockCurrentStateProvider Lifetime: Scoped ImplementationType: MarketAnalyzer.MarketIntelligence.Providers.TsetmcStockCurrentStateProvider': Unable to resolve service for type 'MarketAnalyzer.MarketIntelligence.Application.MarketData.Providers.IProviderSettings' while attempting to activate 'MarketAnalyzer.MarketIntelligence.Providers.TsetmcWebServiceClient'.) (Error while validating the service descriptor 'ServiceType:

MarketAnalyzer.MarketIntelligence.Providers.TsetmcStockOrderBookProvider Lifetime: Scoped ImplementationType: MarketAnalyzer.MarketIntelligence.Providers.TsetmcStockOrderBookProvider': Unable to resolve service for type 'MarketAnalyzer.MarketIntelligence.Application.MarketData.Providers.IProviderSettings' while attempting to activate 'MarketAnalyzer.MarketIntelligence.Providers.TsetmcWebServiceClient'.) (Error while validating the service descriptor 'ServiceType: MarketAnalyzer.MarketIntelligence.Providers.TsetmcStockInvestorActivityProvider Lifetime: Scoped ImplementationType: MarketAnalyzer.MarketIntelligence.Providers.TsetmcStockInvestorActivityProvider': Unable to resolve service for type 'MarketAnalyzer.MarketIntelligence.Domain.IMarketDefinitionStore' while attempting to activate ''MarketAnalyzer.MarketIntelligence.Providers.TseSymbolDirectory'.)

Show more

Ran 3 commands, shared files

خطای DI: یعنی در `Program.cs` سرویس‌های مشترک ثبت نشده‌اند. `IProviderSettings` و `IMarketDefinitionStore` در فازهای ۲ و ۳ تعریف شده‌اند و باید جدا ثبت شوند. کنار `AddMarketDataProviders` این دو خط را هم اضافه کن (ترتیب مهم نیست):

```csharp
builder.Services.AddMarketDataStores();       // Store(فاز ۲) ها: IMarketDefinitionStor
builder.Services.AddMarketDataIngestion();    // هستهٔ ingestion + IProviderSettings +
builder.Services.AddMarketDataProviders(builder.Configuration);
```

با این دو خط، هر سه خطای وابستگی (`IProviderSettings`، `IMarketDefinitionStore`) برطرف می‌شود، چون بقیه زنجیره از همین دو می‌آید.

واحد پول (دلار، تومان، ریال، ...)
الان واحد فقط برای کل بازار تعریف می‌شود (`CurrencyCode` در `MarketDefinitions`). این کافی نیست، چون داخل یک بازار هم ممکن است واحد نمادها فرق کند. مثلاً در Kucoin جفت‌ارز ETH-BTC قیمتش به BTC است. پس دو سطح گذاشتم:

1. **واحد پیش‌فرض بازار** از `ProviderDescriptor` (مثلاً بورس `IRR`).
2. **واحد خاص نماد (اختیاری):** ستون جدید `MarketSymbols.CurrencyCode` . خالی یعنی همان واحد بازار. Provider آن را در `SymbolInfo.CurrencyCode` می‌دهد. در `SymbolRef.CurrencyCode` هم همیشه واحد نهایی می‌آید تا مصرف‌کننده نیازی به حدس‌زدن نداشته باشد.

بازارهای شما با این قاعده:

- بورس و شاخص‌ها: ریال (`IRR`). شاخص «امتیاز» است (`PTS`).
- کریپتوی دلاری: `USDT` برای جفت‌های USDT، و واحد جفت‌های دیگر (مثل BTC) در همان نماد.
- Nobitex و Wallex: تومان (`IRT`).
- واحد جدید بعداً = فقط یک کد جدید. enum یا کد اضافه نمی‌خواهد.

تبدیل واحد برای گزارش در فاز استراتژی (نه ingestion). مدل پیشنهادی من:

- **تومان ↔ ریال:** ضریب ثابت ۱۰.
- **USDT ↔ USD:** فعلاً ۱ به ۱ (قابل‌تغییر در تنظیم).
- **دلار ↔ ریال یا تومان:** از خود کندل‌های بازار Macro (دلار بازار آزاد Tgju) استفاده می‌شود. هر تاریخ با آخرین قیمت پایانی تا همان تاریخ تبدیل می‌شود (as-of)، مثل ضرب سری بورس در نرخ دلار همان روز. جدول نرخ جدا نداریم؛ همان دادهٔ Macro ذخیره‌شده است.
- مقایسهٔ رشد واقعی با ریال برای فولاد و بیت‌کوین، هر دو را به یک واحد می‌برد (مثلاً دلار) و بعد base 100 می‌کند.

اگر مخالف نرخ دلار از Tgju هستی (مثلاً نرخ نیما یا دلار دیگر)، بگو که «کدام سمبل نرخ مرجع است» را در `MarketSettings` قابل‌انتخاب کنیم.

پنج فایل جایگزین (zip ضمیمه) `MarketSymbol` ، `MarketSymbolConfiguration` ، `SymbolInfo` ، `SymbolRef` و `EfMarketSymbolStore` هستند. چون ستون جدید داریم، Migration دوم لازم است:

```
dotnet ef migrations add MarketData_SymbolCurrency -p MarketAnalyzer.Persistence -
s MarketAnalyzer.Api
```

فقط باید `AddColumn CurrencyCode nvarchar(10) null` روی `MarketSymbols` داشته باشد. چیز دیگری در آن دیدی، نفرست و به من بگو.

بعد از رفع خطای DI و build سبز، ۴-ج (کریپتو) را شروع می‌کنم. سؤال قبلی‌ام هنوز باز است: Nobitex و Wallex یک بازار `crypto-toman` با دو منبع قابل‌تعویض باشند؟

آره قابل تعویض باشن یعنی یکدوم رو باید سیستم بگیره
کلا سیستم اینطوریه که از یک مرجع باید بگیره ولی من باید بتونم بگم مثلا اگه چند تا سورس بود

تبدیل واحد هم اگه قراره داینامیک باشه ممکنه نخوام از tgju بگیرم از تتر نوبیتکس یا والکس بگیرم
چون ممکنه یکدوم قطع بشه
اگه اوکیه ادامه بده

Ran 6 commands, read memory

الآن Phase 4c (کریپتو، ماکرو، KuCoin) نوشته شده؛ قبل از زیپ‌کردن، سازگاری نام‌ها را با تایپ‌های موجود چک می‌کنم.

Ran 4 commands

Stopped reading memory

Saving decisions to memory, then sending the zip.

Read memory, shared files

الان چکار کنم؟

Added to memory

الان این کارها را بکن:

1. **فایل** `MarketIntelligence-Phase4c.zip` **را باز کن** و روی ریشهٔ `backend` اکسترکت کن (مسیرها داخل زیپ کامل است و فایل‌های قبلی را بازنویسی می‌کند).
2. Providers را **به sln اضافه کن** (اگر قبلاً نکرده‌ای):
```
dotnet sln MarketAnalyzer.sln add MarketIntelligence/src/MarketAnalyzer.MarketIntelligence.Providers/MarketAnalyzer.MarketIntelligence.Providers.csproj
```
3. **بیلد بگیر** و خطاها را بفرست. کد در sandbox کامپایل نشده، پس ممکن است چند خطای کوچک داشته باشد.
4. اگر خطای DI قبلی (`IProviderSettings` / `IMarketDefinitionStore`) را هنوز داری، در `Program.cs` این دو را صدا بزن: `AddMarketDataStores()` و `AddMarketDataIngestion()` (و `AddMarketDataProviders(configuration)`). بعد بگو درست شد یا نه.
5. **مایگریشن** `MarketData_SymbolCurrency` را چک کن: باید فقط ستون `CurrencyCode nvarchar(10) null` را به `MarketSymbols` اضافه کند. اگر چیز دیگری داشت، بفرست.

محتوای این فاز:

- پایهٔ مشترک UDF (`UdfCandleClientBase`)؛ نهایت‌نگر روی آن منتقل شد و خطاها دیگر به «لیست خالی» تبدیل نمی‌شوند.
- کریپتو-تومان: Wallex (پیش‌فرض) و Nobitex قابل‌تعویض، با فهرست ثابت و کلید استاندارد مثل `BTC-IRT`؛ ارز هر نماد جدا ثبت می‌شود (IRT یا USDT).
- ماکرو (Tgju): فهرست ثابت دلار، سکه، طلا، انس (USD) و حباب‌ها (درصد).
- KuCoin: فهرست و کندل، با تنظیم `QuoteCurrencies` (پیش‌فرض فقط USDT؛ کد قدیمی همهٔ ~هزار جفت‌ارز را می‌گرفت).
- باگ قدیمی: Nobitex آدرس Wallex را می‌خواند؛ اصلاح شد. تبدیل ×۱۰ تومان به ریال هم حذف شد و واحد از `CurrencyCode` خوانده می‌شود.

تعویض Wallex به Nobitex: در `MarketSettings` برای scope `market:crypto-toman` مقدار `Provider:HistoricalCandles` را `nobitex` بگذار. این مرحلهٔ تأییدیه و پاک‌کردن کندل‌ها در فاز ۵ می‌آید.

فاز بعدی: هنوز Kilid، IME (املاک)، شاخص‌های بورس و صندوق‌ها (Fipiran) مانده. پیشنهادم این است که اول فاز ۵ (API، Bootstrap، scheduler تنظیمات، تعویض منبع) را بسازم، چون بدون آن هیچ Provider اجرا نمی‌شود، و این ۴ منبع آخر را بعداً اضافه کنم. اگر موافقی بگو «فاز ۵».

ادامه بده

فاز ۵ را شروع می‌کنم: اول زمان‌بند و Bootstrap؛ ابتدا ساختار فعلی و زمان‌بند قدیمی را می‌خوانم.

Ran 8 commands

This response didn’t load.

ادامه بده

Ran 6 commands

در حال بسته‌بندی فاز ۵ (زمان‌بند، Bootstrap، تعویض منبع، API) هستم.

Ran a command

Updated memory, shared files

فاز ۵ ارسال شد: `MarketIntelligence-Phase5.zip`. کامپایل نشده؛ بیلد را بگیر و خطاها را بفرست.

۱. نصب

- زیپ را روی ریشهٔ `backend` اکسترکت کن. فایل‌های Operations داخل Application می‌روند و `MarketDataSettingsController` داخل `MarketAnalyzer.Api`.
- در `Program.cs` به‌جای `AddMarketCollectors()` و `AddMarketCollectorRunner()` این‌ها را بگذار:

```csharp
builder.Services.AddMarketDataStores();
builder.Services.AddMarketDataIngestion();
builder.Services.AddMarketDataProviders(builder.Configuration);
builder.Services.AddMarketDataOperations(builder.Configuration);
```

- هر دو را همزمان نگه نداری؛ زمان‌بند قدیمی و جدید با هم روی دیتابیس می‌افتند.

۲. چه چیزی ساخته شد

- **Bootstrap:** تنظیمات پیش‌فرض را فقط اگر وجود نداشته باشند در Provider هنگام شروع، ردیف بازارها را از `MarketSettings`، Wallex می‌نویسد، پس تغییرات تو بازنویسی نمی‌شود. برای `crypto-toman` که دو منبع دارد پیش‌فرض می‌شود.
- **زمان‌بند عمومی:** برای هر (بازار × کار) از روی `MarketSettings` تصمیم می‌گیرد.
  - کارها: همگام‌سازی فهرست نمادها، کندل، وضعیت لحظه‌ای، صف سفارش، حقیقی/حقوقی.
  - اجرای همزمان یک کار برای یک بازار ممنوع است.
  - شکست با backoff نمایی تا سقف یک ساعت دوباره تلاش می‌شود.
  - قفل برنامه (`ClientExposedException`) شکست حساب نمی‌شود و ۳۰ ثانیه بعد دوباره امتحان می‌شود.
  - اگر ۱۰ نماد اول کندل همه شکست بخورند، اجرا قطع می‌شود چون منبع احتمالاً از دسترس خارج است.
  - کلید `DisableTasks=true` همه را متوقف می‌کند.
- **تعویض منبع:** `SwitchProvider` برای کندل‌ها بدون `confirmWipe=true` کاری نمی‌کند. با تأیید، کندل‌های آن بازار پاک می‌شود و اجرای بعدی از منبع جدید کامل می‌گیرد. اگر همان لحظه دریافت کندل در جریان باشد، `Busy` برمی‌گرداند.
- **API (فقط `sys_admin`):** `Overview`، `ProviderSettings`، `SetSetting`، `SwitchProvider` و `RunJob` برای اجرای دستی. کلیدهای `SetSetting` و مقدارهای محرمانه را قبول نمی‌کند `Provider:*`.

- کندل ناتمام امروز در (crypto مثل) **لحظه‌ای Provider وضعیت امروز برای بازارهای بدون** `CurrentSymbolStates` ذخیره می‌شود، نه در تاریخچه.

۳. فرض‌هایی که خودم گذاشتم (در `MarketSettings` قابل‌تغییر)

- ساعات بورس ۰۸:۴۵ تا ۱۳:۰۰ تهران، شنبه تا چهارشنبه، برای وضعیت لحظه‌ای و صف سفارش (هر ۱ دقیقه).
- حقیقی/حقوقی ۰۹:۰۰ تا ۱۹:۰۰ هر ۳۰ دقیقه.
- کندل هر ۶ ساعت و فهرست نمادها روزی یک‌بار.
- کلیدها به شکل `Scheduler:{Job}:IntervalSeconds` و `Enabled` / `WindowTo` / `WindowFrom` هستند.

۴. محدودیت‌های شناخته‌شده

- قفل «اجرای همزمان» فقط داخل یک پروسه معتبر است. اگر چند نمونه از برنامه اجرا کنی (یا کنار RabbitMQ چند consumer)، باید `sp_getapplock` اضافه شود. بگو اگر لازم است.
- تعطیلات رسمی بورس بررسی نمی‌شود؛ فقط روزهای پنجشنبه و جمعه. روز تعطیل فقط دادهٔ تکراری می‌گیرد و ضرری ندارد.
- بازار `tse-index` هنوز فهرست نماد ندارد، پس کندلش تا اضافه‌شدن Provider فهرست شاخص‌ها اجرا نمی‌شود.

مراحل بعد: Kilid، IME، فهرست شاخص‌ها و صندوق‌ها (Fipiran) در Phase 4d، بعد خواننده‌ها (فاز ۶) و پاکسازی (فاز ۷). اول کدام؟

فاز 4d چیه؟ادامه providerها؟

بله، «فاز 4d» ادامهٔ Providerها است؛ اسم را خودم گذاشتم (4a و 4b و 4c قبلاً بودند). چهار منبع باقی‌مانده را می‌گیرد:

- **شاخص‌های بورس (`tse-index`):** فهرست نماد شاخص‌ها از `TseTmcIndex.json`. کندل آن از قبل با نهایت‌نگر هست ولی بدون فهرست اجرا نمی‌شود. در کد قدیمی `GetAllIndexAsync` یک تاریخ ثابت (`20260608`) و یک حلقهٔ ۴ بار تکراری داشت که درستش می‌کنم.
- **IME (بورس کالا):** فهرست و کندل.
- **املاک (Kilid):** کندل ماهانهٔ بومی، حدود ۸۴۷ منطقه.
- **صندوق‌ها (Fipiran):** NAV صندوق‌ها.

اولویتم این است که **اول شاخص‌ها** را بسازم، چون کوچک است و `tse-index` را فعال می‌کند. بعد Kilid، IME و صندوق‌ها.

اگر موافقی همین ترتیب را شروع می‌کنم. اگر یکی از این‌ها برایت مهم‌تر است (مثلاً صندوق‌ها)، همان را اول بگو.

اوکی هست شروع کن

شروع فاز 4d با شاخص‌های بورس: اول کد قدیمی دریافت شاخص‌ها و فایل `TseTmcIndex.json` را می‌خوانم.

Ran 6 commands

Still on 4d (شاخص‌ها): دارم می‌بینم کد قدیمی کندل شاخص را از کجا می‌گرفت.

Ran 8 commands, shared files

فاز 4d را فرستادم: `MarketIntelligence-Phase4d.zip`. هنوز کامپایل نشده، پس بیلد را بگیر و خطاها را بفرست. زیپ فاز ۵ را هم دوباره فرستادم، چون یک کار جدید به آن اضافه شده است.

محتوای 4d

- **شاخص‌های بورس (`tse-index`):**
  - ۶۴ شاخص از فهرست ثابت (`Resources/tse-index.symbols.json`).
  - کلید هر شاخص `InstrumentId` است و اگر نداشت `InsCode`، مثل کد قدیمی.
  - کندل شاخص‌ها از قبل با نهایت‌نگر آماده بود و حالا با این فهرست فعال می‌شود.
  - کشف خودکار شاخص‌ها از SOAP را نیاوردم، چون تاریخ ثابت و حلقهٔ تکراری داشت. اگر لازم داری، جدا اضافه می‌کنم.
- **IME، املاک (Kilid) و صندوق‌ها (Fipiran):** فهرست و کندل برای IME و Kilid، و فهرست و NAV برای Fipiran. فهرست‌های ثابت IME و مناطق Kilid داخل DLL هستند (`ime.symbols.json`، `real-estate.symbols.json`).
- **ثبت در DI:** همه در `ProvidersRegistration` ثبت شده‌اند.

تغییر فاز ۵: کار `RefreshFundNav` اضافه شد (هر ۶ ساعت)، چون Fipiran بدون آن هیچ‌وقت NAV ذخیره نمی‌کرد.

الان انجام بده

1. هر دو زیپ را روی `backend` اکسترکت کن؛ 4d را اول و فاز ۵ را بعد.
2. بیلد بگیر و خطاها را بفرست.
3. اگر مایگریشن جدیدی لازم شد (مثلاً بعد از تغییرات قبلی)، بگو تا بررسی کنم.

بعدش: فاز ۶ (خواننده‌ها مثل `IAdjustedCandleReader`)، بعد فاز ۷ (پاکسازی کد قدیمی). فاز ۶ را شروع کنم؟

خب الان بیلد اوکی بود
ران هم میشه باید دیتا بگیره الان یا هنوز نه؟

بله، باید همین الان دیتا بگیرد، به شرط این‌که این چند مورد برقرار باشد.

پیش‌شرط‌ها

1. **مایگریشن اعمال شده باشد** (`dotnet ef database update`)، تا جدول‌های schema `marketintelligence` وجود داشته باشند.
2. **در `Program.cs` همین چهار خط باشد و Collectorهای قدیمی برداشته شده باشند:** `AddMarketDataStores()`، `AddMarketDataIngestion()`، `AddMarketDataProviders(configuration)`، `AddMarketDataOperations(configuration)`. اگر زمان‌بند قدیمی هم فعال بماند، دو زمان‌بند روی همان دیتابیس می‌افتند.
3. `UrlSettings` **هنوز باید bind شود.** پیش‌فرض آدرس‌ها فعلاً از `IOptions<UrlSettings>` می‌آید. اگر این bind را فقط داخل `AddMarketCollectors()` داشتی و حذفش کردی، آدرس‌ها خالی می‌مانند و کندل‌ها `Failed` می‌شوند. در این حالت این را جدا اضافه کن:
   ```
   builder.Services.Configure<UrlSettings>
   (builder.Configuration.GetSection("UrlSettings"))
   ```
   اسم بخش را مطابق همان فایل قدیمی بگذار.
4. **اعتبار TSETMC** (نام کاربری و رمز) در User Secrets باشد، وگرنه فهرست سهام و وضعیت لحظه‌ای بورس شکست می‌خورد.

چه اتفاقی می‌افتد

- بعد از استارت، Bootstrap بازارها و تنظیمات پیش‌فرض را در `MarketSettings` می‌نویسد.
- بعد فهرست نمادها همگام می‌شود و پس از آن کندل‌ها دریافت می‌شوند.
- اجرای اول کندل برای هر نماد تاریخچهٔ کامل (تا ۱۰ سال) می‌گیرد؛ با چند هزار نماد بورس و ۴ نماد همزمان ممکن است ساعت‌ها طول بکشد. اجرای بعدی فقط ۱۰۰ کندل آخر را می‌گیرد.

پیشنهاد برای اولین تست: همه را یک‌جا روشن نکن.

- بازارهای کوچک را اول امتحان کن، مثل `macro` (چند ده نماد) یا `crypto-toman` .
- از `POST MarketDataSettings/RunJob` با `{"marketKey":"macro","job":2}` استفاده کن. `job` برای کندل ۲ و برای فهرست نمادها ۱ است.
- برای بازارهایی که نمی‌خواهی بالا بیایند، در `MarketSettings` برای scope `market:{key}` مقدار `Scheduler:IngestCandles:Enabled` را `false` کن.

چطور بفهمی کار می‌کند

- در لاگ دنبال خطهایی مثل `IngestCandles macro: processed=… updated=… failed=…` بگرد. اگر `failed` زیاد بود، خطای نمونه همان‌جا چاپ می‌شود.
- در دیتابیس بررسی کن که ردیف‌های `MarketSymbols` و `HistoricalCandles` پر شده باشند (نام جدول‌ها ممکن است کمی با این‌ها فرق داشته باشد).

اولین چیزی که از لاگ یا جواب `RunJob` دیدی را بفرست تا اگر خطایی بود، همان‌جا درستش کنم.

[Warning-10:51:48.528] No store type was specified for the decimal property 'StatisticalNav' on entity type 'InvestmentFundInfo'. This will cause values to…

PASTED

همهٔ Providerها دوبار شمرده می‌شوند (مثلاً `crypto-toman` با ۴ کاندید برای ۲ منبع). علت را در ثبت DI می‌بینم.

Ran 2 commands, shared files

لاگ نشان می‌دهد هر Provider دوبار در DI ثبت شده است. همهٔ شمارش‌ها دقیقاً دو برابرند: `crypto-toman` با ۴ کاندید برای ۲ منبع، و `ime` با «۲ پیش‌فرض» برای ۱ Provider. پس Resolver و Bootstrap فکر می‌کنند چند منبع هست و هیچ‌کدام را انتخاب نمی‌کنند. به همین دلیل هیچ کاری اجرا نشد.

علت: `AddMarketDataProviders` دو بار صدا زده شده است. احتمالاً یک‌بار در `Program.cs` و یک‌بار داخل یک Extension دیگر.

اصلاح: فایل `MarketIntelligence-Phase5-hotfix1.zip` را روی `backend` اکسترکت کن؛ فقط `ProvidersRegistration.cs` است. بار دوم این متد حالا هیچ کاری نمی‌کند. کد را کامپایل نکرده‌ام، پس بیلد را هم بگیر.

کار تو:

1. بعد از اعمال زیپ، برنامه را دوباره اجرا کن. Bootstrap دفعهٔ بعد منبع پیش‌فرض را در `MarketSettings` می‌نویسد. چیزی را پاک نکن؛ مقدارهای قبلی بی‌ضرر است.
2. دنبال ردیف دوم این فراخوانی بگرد و حذفش کن، تا ثبت‌ها تمیز بماند.
3. لاگ بعدی را بفرست. باید خط‌هایی مثل `SyncCatalog macro: processed=…` ببینی.

هشدار اول لاگ: `StatisticalNav ... InvestmentFundInfo` مربوط به موجودیت قدیمی است و در فاز ۷ همراه آن حذف می‌شود، پس الان کاری با آن نداشته باش.

[Warning-10:54:33.144] No store type was specified for the decimal property 'StatisticalNav' on entity type 'InvestmentFundInfo'. This will cause values to…

PASTED

Ran 5 commands, shared files

خطا از دیتابیس است: ستون `CurrencyCode` جدول `marketintelligence.MarketSymbols` در دیتابیس `NOT NULL` است. چون نمادهای بازار `crypto-toman` ارز همان بازار را دارند، مقدارشان `null` می‌شود و درج با خطای ۵۱۵ رد می‌شود. این ۴ خط اول لاگ (قبل از خطا) نشان می‌دهند که Bootstrap و انتخاب منبع حالا درست کار می‌کند.

اصلاح: `MarketIntelligence-Phase5-hotfix2.zip` را روی `backend` اکسترکت کن. در سه فایل Configuration، ستون‌های اختیاری صریحاً `IsRequired(false)` شدند: `CurrencyCode`، `ExternalId`، `Name`، `PersianName`، `MetadataJson`، `UpdatedBy` و `Description`. کامپایل نشده، پس بیلد را هم بگیر.

چون خود دیتابیس هم باید عوض شود:

1. بیلد کن و یک مایگریشن جدید بساز، مثلاً `dotnet ef migrations add MarketData_NullableColumns`، و فایلش را نگاه کن. باید `AlterColumn ... nullable: true` برای همین ستون‌ها داشته باشد؛ بعد `database update`.
2. اگر مایگریشن خالی ساخته شد (یعنی snapshot فکر می‌کند ستون‌ها از قبل nullable هستند ولی دیتابیس نیست)، این SQL را مستقیم اجرا کن:

```sql
ALTER TABLE marketintelligence.MarketSymbols ALTER COLUMN ExternalId nvarchar(64) NU
ALTER TABLE marketintelligence.MarketSymbols ALTER COLUMN CurrencyCode nvarchar(10)
ALTER TABLE marketintelligence.MarketSymbols ALTER COLUMN [Name] nvarchar(200) NULL;
ALTER TABLE marketintelligence.MarketSymbols ALTER COLUMN PersianName nvarchar(200)
ALTER TABLE marketintelligence.MarketSymbols ALTER COLUMN MetadataJson nvarchar(max)
```

اگر `MarketSettings.UpdatedBy` یا `CorporateActions.Description` هم NOT NULL بودند، همان کار را برایشان بکن.

علت دقیق را نمی‌دانم: نمی‌دانم چرا مایگریشن قبلی ستون را NOT NULL ساخته. اگر برایت مهم است، فایل مایگریشن `MarketData_SymbolCurrency` را بفرست تا نگاه کنم.

بعد از اصلاح، لاگ بعدی را بفرست. کار `SyncCatalog` برای `crypto-toman` الان با backoff دوباره تلاش می‌کند؛ با ری‌استارت زودتر اجرا می‌شود.

```
public partial class AddPhase5 : Migration
{
  /// <inheritdoc />
  protected override void Up(MigrationBuilder migrationBuilder)
```

```
        {
            migrationBuilder.AddColumn<string>(
                name: "CurrencyCode",
                schema: "marketintelligence",
                table: "MarketSymbols",
                type: "nvarchar(max)",
                nullable: false,
                defaultValue: "");
        }

        /// <inheritdoc />
        protected override void Down(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.DropColumn(
                name: "CurrencyCode",
                schema: "marketintelligence",
                table: "MarketSymbols");
        }
    }
```

Show more

این مایگریشن نشان می‌دهد که ستون با `nvarchar(max)` و `nullable: false` ساخته شده. در مدل من برای این ستون `HasMaxLength(10)` و nullable بودن تعریف شده، پس مایگریشن را از یک نسخهٔ ناقص مدل ساخته‌ای. احتمالاً `MarketSymbol.cs` (Entity یا Configuration آن لحظه هنوز نسخهٔ جدید، `MarketSymbolConfiguration.cs` از زیپ currency) نبوده است.

بهترین راه (تمیز): همین مایگریشن را حذف و دوباره بساز. دیتابیس دست‌نخورده نمی‌ماند، ولی چون در ستون هیچ دادهٔ مفیدی نیست، خطری ندارد.

1. مدل را کامل کن: زیپ `Phase4b-currency` و بعد `Phase5-hotfix2` باید اعمال شده باشند. تأیید کن که `MarketSymbol.CurrencyCode` از نوع `string?` است و در Configuration خط `HasMaxLength(10).IsRequired(false)` هست.
2. برگرد به مایگریشن قبل از این:
   `dotnet ef database update <نام مایگریشن قبل از AddPhase5>`

3. `dotnet ef migrations remove`
4. `dotnet ef migrations add AddPhase5`
5. `dotnet ef database update`

مایگریشن جدید باید دقیقاً همین باشد:

```csharp
migrationBuilder.AddColumn<string>(
    name: "CurrencyCode", schema: "marketintelligence", table: "MarketSymbols",
    type: "nvarchar(10)", maxLength: 10, nullable: true);
```

اگر چیز دیگری هم داشت (مثلاً `AlterColumn` برای `Name` یا `ExternalId`)، یعنی دیتابیس آن ستون‌ها را هم NOT NULL ساخته بود و درست است که باقی بماند.

راه سریع‌تر: اگر نمی‌خواهی تاریخچه را تمیز کنی، فقط یک مایگریشن جدید بساز (`dotnet ef migrations add Phase5_CurrencyNullable`)؛ باید `AlterColumn` به `nvarchar(10)` و `nullable: true` تولید کند. نتیجهٔ نهایی همان است.

بعد از اعمال، برنامه را اجرا کن و لاگ بعدی را بفرست. باید `SyncCatalog crypto-toman: processed=…` ببینی.

تو کلاس MarketSymbol من خودم این رو اضافه کردم
public string CurrencyCode { get; set; }
نبود و خطا میداد
الان این رو nullable بزارم و ماگریشن بزنم؟

بله، `string?` بگذار و مایگریشن بزن. ولی فایل `MarketSymbol.cs` را با نسخهٔ زیپ جایگزین کن، چون آن نسخه `CurrencyCode` را دارد و باقی فایل‌ها (`MarketSymbolConfiguration`، `EfMarketSymbolStore`) به آن وابسته‌اند. اگر زیپ‌های `Phase4b-currency` و `Phase5-hotfix2` را اعمال نکرده بودی، همین علت اصلی ناقص بودن مدل بود.

مراحل:

1. `MarketSymbol.cs` را از زیپ `Phase4b-currency` بردار. باید این خط را داشته باشد:
   `public string? CurrencyCode { get; set; }`
   اگر خودت خط `public string CurrencyCode` را اضافه کرده‌ای، حذفش کن تا دو تعریف نشود.
2. `MarketSymbolConfiguration.cs`، `MarketSettingConfiguration.cs` و `CorporateActionConfiguration.cs` را از `Phase5-hotfix2` بردار.
3. بیلد بگیر.
4. مایگریشن `AddPhase5` قبلی را اگر اعمال شده، به‌صورت تمیز برگردان (`database update <قبلی>` و `migrations remove`) یا فقط یک مایگریشن جدید بساز.
5. مایگریشن جدید را باز کن؛ باید `type: "nvarchar(10)"` و `nullable: true` باشد.
6. `dotnet ef database update` را بزن.

اگر مایگریشن جدید `nvarchar(max)` **یا** `nullable: false` **نشان داد**، یعنی هنوز یکی از فایل‌های بالا اعمال نشده یا نسخهٔ قدیمی کامپایل شده؛ دوباره بیلد کن و دوباره بساز.

بعد از اعمال، برنامه را اجرا کن و لاگ بعدی را بفرست.

[Warning-11:05:55.864] No store type was specified for the decimal property 'StatisticalNav' on entity type 'InvestmentFundInfo'. This will cause values to...

PASTED

دو مشکل در این لاگ هست: خطای `Sequence contains no matching element` در کندل (باگ خودم) و 405 در Fipiran. اول هر دو را در کد بررسی می‌کنم.

Ran 6 commands

دارم باگ کندل را درست می‌کنم و برای Fipiran خطای قابل‌عیب‌یابی اضافه می‌کنم.

Ran 4 commands, shared files

فایل `MarketIntelligence-Phase5-hotfix3.zip` را فرستادم. کامپایل نشده، پس بیلد را بگیر. در لاگ دو مشکل بود.

۱. خطای کندل (`Sequence contains no matching element`) — باگ من بود.

- DI ها فقط به‌صورت `IMarketDataProvider` در Provider ثبت می‌شوند. `GetServices<IHistoricalCandleProvider>()` خالی برمی‌گشت، `First` استثنا می‌داد و کل اجرای بازار می‌ترکید. این همان چیزی بود که برای `tse`، `tse-index` و `crypto-toman` دیدی.
- `IMarketDataProvider` حالا از فهرست Provider فیلتر می‌شود. اگر پیدا نشد، فقط همان نماد شکست می‌خورد، نه کل بازار.
- پیش‌نیاز کندل برای این سه بازار برقرار بود (فهرست نمادها همگام شده بود)، پس بعد از این اصلاح باید کندل بگیرند.

۲. Fipiran: خطای 405 (Method Not Allowed) روی فهرست صندوق‌ها.

- نمی‌توانم از اینجا به fipiran.ir وصل شوم و تست کنم. در کد قدیمی هم خطا بلعیده می‌شد، پس معلوم نیست آن آدرس واقعاً کار می‌کرده.
- در این نسخه متد و بدنه قابل‌تنظیم شدند و پیام خطا شروع بدنهٔ پاسخ سرور را هم دارد، تا علت دیده شود.
- اگر 405 ادامه داشت، این‌ها را بفرست:
  - در مرورگر صفحهٔ صندوق‌ها را در fipiran.ir باز کن، F12 و Network.
  - درخواستی را که فهرست صندوق‌ها را می‌آورد پیدا کن و آدرس کامل (URL) و متد (GET یا POST) و در صورت POST بدنه‌اش را بفرست.
- بعد با `SetSetting` روی scope `provider:fipiran` این کلیدها را عوض می‌کنی، بدون انتشار دوباره:
  - `FundsUrl`
  - `FundsMethod` (مقدار `GET` یا `POST`)
  - `FundsBody`
- اگر مقدارهای قبلی `FundsUrl` از قبل در `MarketSettings` ذخیره شده، فقط با `SetSetting` عوض می‌شود. Bootstrap مقدار موجود را بازنویسی نمی‌کند.

چیزی که از لاگ برایم روشن نیست: بقیهٔ کارها (`SyncCatalog` برای بازارهای دیگر) خطا نداشتند ولی لاگ موفقیت‌شان را هم ندیدم. بعد از اعمال hotfix3، لاگ بعدی را بفرست تا ببینم کندل واقعاً ذخیره می‌شود. اگر `Failed` زیاد بود، نمونهٔ خطا همان‌جا چاپ می‌شود.